

Introduction to Exercises

The practical exercises in this course are intended to complement the technical training material. While some questions introduce additional concepts and language features covered during the course, most are designed to reinforce what has already been demonstrated.

Each exercise outlines its objectives and highlights any specific tools or requirements needed to complete it. In some cases, you will need to consult online documentation or manual pages—this is a normal and expected part of the problem-solving process.

Optional tasks may appear at the end of a section under “*Stretch objectives (Things to try)*”. These are typically more challenging or explore related tools and features. You are not generally expected to complete them unless you finish the main exercises within the allocated time. Hints may also be provided where appropriate.

Many exercises will involve writing substantial code snippets or even complete programs, though these are usually relatively concise. Where necessary, supporting materials—such as data files, boilerplate code, pre-written functions, or test runners—will be provided, and the instructions will clearly indicate when these are included.

If the course is carried out on QA premises, you have the option to use Microsoft Windows or a Linux VM. The username and password for the Linux VM is qa.

Complexity key

Each exercise has a designated level of challenge, reflecting in the use of these friendly icons:

		
Easy A straightforward linear-style end-to-end problem	Medium difficulty A problem with some unique features, requiring deeper thought	Harder difficulty A problem with many challenges, often requiring additional topic or technical research

Note. Assigned target times do not necessarily reflect the complexity level of the task.

Online Directory Structure

On a Microsoft Windows platform all of the files mentioned here are stored in a directory called **exercises**. This is normally on your C: drive, but may reside elsewhere, particularly for courses not carried out on QA premises. Your instructor should be able to clarify their exact location.

On Linux: all the files should be in your home directory. If using the supplied Linux VM, this will be **/home/qa/exercises**.

Solutions

Sample solutions for all questions are available online in a subdirectory named *solutions*.

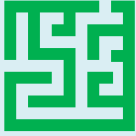
Keep in mind that, as with most programming tasks, there are multiple valid approaches to solving a problem. If your solution differs from the provided one, it does not necessarily mean it is incorrect. There is rarely a single “model” answer—only solutions with their own strengths and weaknesses.

Software engineering also involves a degree of creativity, so some level of subjective interpretation is expected. You are encouraged to discuss different approaches with your instructor and your peers.

While you can review and run the provided solutions without attempting the exercises yourself, doing so may reduce the overall benefit you gain from the course. If you choose not to attempt a question, make sure you fully understand the solution.

Intermediate Python: Exercise #1

Python 3 Revision

Complexity level	Solo, Pair or Team?	Target time
		 30 minutes

Brief

This exercise presents a problem for you to solve using Python. It is intended to reinforce your foundational knowledge and support your preparation for the wider challenges of the programme.

Throughout the process, you are encouraged to share and discuss ideas with your peers, particularly your understanding of the given problem.

After completing the task, don't forget to seek constructive feedback from your instructor.

Comparing your solution with those of your peers is also strongly recommended as different techniques and approaches may come to light which may prove useful to you.

Tools overview

You will be using a variety of software tools:

- **Python 3** (you are recommended to use at least Python version 3.12)
 - a high-level, general purpose programming language favoured by Data Scientists for experiments
 - <https://www.python.org/>
- **A suitable IDE**, e.g.
 - Microsoft Visual Studio Code
 - PyCharm Community Edition
 - IDLE
- **"iban.csv" – a CSV (comma separated value) file**
 - Which contains simulated International Bank Account Number (IBAN) data (some valid, some not...)
 - is required to solve this problem
 - is located in "exercises" folder or your home folder (Trainer will advise)

Solutions should be tested on a Microsoft Windows platform but should work, largely unaltered on others, e.g. Linux distributions, Apple Mac OS X etc., including through the use of containerisation and virtualisation.

Expected outcome(s)

On success, you should be able to confidently solve a problem by writing appropriately structured and idiomatic Python 3 code which reflects good practice and follows PEP8 guidelines.

Set Problem

Locate a text file called “iban.csv” which contains IBAN entries stored in the following format:

```
part1,part2,part3,part4,part5,part6
GB90,SPQQ,2081,2191,3619,39
GB08,QTAR,3863,7940,2654,23
GB10,OQOD,3315,0983,9301,03
GB27,UDAX,4332,1819,6001,33
<snip>
```

Create a function-based Python solution that reads this CSV file containing 20 sample IBANs split across 6 fields of lengths 4, 4, 4, 4, 4, 2.

Reassemble each row into a full IBAN, validate it using the IBAN Mod-97 algorithm, and write the results into two output files: `valid.txt` and `invalid.txt` accordingly.

Then test the solution.

Technical notes

A UK IBAN has a fixed length of 22 characters and follows this structure:

GBkk BBBB SSSS SCCC CCCC CC

Where:

- GB → Country code (always letters)
- kk → Check digits (numbers)
- BBBB → Bank identifier (letters)
- SSSSSS → Sort code (numbers)
- CCCCCCCC → Account number (numbers)

Sources:

Ref. ISO 13616 (IBAN standard)

- Defines IBAN structure globally
- Country-specific formats (like UK) are defined within this

Ref. SWIFT IBAN Registry

- Official IBAN format registry

IBAN Mod-97 algorithm

This is a validation check that is used by banks to make sure an IBAN hasn't been mistyped, i.e. digits haven't been accidentally transposed etc. If the IBAN is valid, the calculation will always give a result of 1.

Note. This doesn't ensure it is legitimately "real", just that the IBAN is correctly structured.

The step-by-step process you will need to duplicate is:

1. Take the IBAN string
2. Remove any spaces (leading, embedded or trailing)
3. Move the first 4 characters to the end of the string
4. Replace all letters in the strings with numbers, e.g.

A = 10

B = 11

C = 12

...

Z = 35

This generates a long number (- you need to think about how to store and process this).

5. Take this long number and perform a modulo division by 97 – we only care about the remainder, of course.
6. If that remainder is a 1, the IBAN is valid.

Stretch objectives (Things to try)

1. Ensure that your code is PEP8 compliant by using your IDE or available tooling.
 2. Include suitable error that prevents the script from crashing if the file cannot be found/opened (e.g. due to permissions) etc. Correctly report any errors to `stderr`, rather than `stdout`.
 3. Use an appropriate linting tool to discover any potential problems with your code, e.g. unnecessary imports, unused variables etc.
-

Additional notes and reflection points

- Which data structures did you select as part of your solution?
- Did you find a better solution when comparing your solution with your peers?
- Which part of the solution did you find the most complex to solve?
- Did you choose EAFP or LBYL error handling techniques?

Intermediate Python: Sample Solution #1

Python 3.12 version

Below is a sample Python script illustrating some of the key steps from the practical. There are multiple ways to approach this task, so your code does not need to match exactly, but it should be clear, well-structured, and PEP 8 compliant. And, hopefully, solve the problem as set.

```
from __future__ import annotations
import csv
from pathlib import Path

def reassemble_iban(parts: list[str]) -> str:
    """
    Reassemble an IBAN from 6 CSV fields of lengths 4, 4, 4, 4, 4, 2.
    """
    cleaned = [part.strip().upper() for part in parts]
    if len(cleaned) != 6:
        raise ValueError("Each row must contain exactly 6 fields.")
    expected_lengths = [4, 4, 4, 4, 4, 2]
    if [len(part) for part in cleaned] != expected_lengths:
        raise ValueError("IBAN row parts do not match the expected 4,4,4,4,4,2
layout.")
    iban = "".join(cleaned)
    if not iban.isalnum():
        raise ValueError("IBAN contains invalid characters.")
    return iban

def iban_to_numeric_string(iban: str) -> str:
    """
    Rearrange the IBAN and convert letters to numbers for mod-97 checking.
    """
    rearranged = iban[4:] + iban[:4]
    converted: list[str] = []
    for char in rearranged:
        if char.isdigit():
            converted.append(char)
        else:
            converted.append(str(ord(char) - ord("A") + 10))
    return "".join(converted)

def perform_mod97(number_string: str) -> int:
    """
    Calculate modulo 97 incrementally to avoid large integer issues.
```

```

"""
remainder = 0
for char in number_string:
    remainder = (remainder * 10 + int(char)) % 97
return remainder

def is_valid_iban(iban: str) -> bool:
    """
    Return True when an IBAN passes the mod-97 check.
    """
    if len(iban) < 5 or not iban.isalnum():
        return False
    numeric = iban_to_numeric_string(iban)
    return perform_mod97(numeric) == 1

def process_iban_file(
    input_csv: str | Path,
    valid_output: str | Path = "valid.txt",
    invalid_output: str | Path = "invalid.txt",
) -> tuple[list[str], list[str]]:
    """
    Read a CSV of 6-part IBAN rows, validate each IBAN, and write
    valid and invalid entries to separate text files.
    """
    valid_ibans: list[str] = []
    invalid_ibans: list[str] = []

    with Path(input_csv).open("rt", newline="", encoding="utf-8") as handle:
        reader = csv.reader(handle)
        header = next(reader, None)  # Discard for now
        for row in reader:
            try:
                iban = reassemble_iban(row)
                if is_valid_iban(iban):
                    valid_ibans.append(iban)
            except ValueError:
                invalid_ibans.append("".join(cell.strip().upper() for cell in row))

    Path(valid_output).write_text("\n".join(valid_ibans) + ("\n" if valid_ibans else
    ""), encoding="utf-8")
    Path(invalid_output).write_text("\n".join(invalid_ibans) + ("\n" if invalid_ibans
    else ""), encoding="utf-8")

    return valid_ibans, invalid_ibans

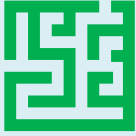
def main() -> None:
    valid_ibans, invalid_ibans = process_iban_file("iban.csv", "valid.txt",
    "invalid.txt")
    print(f"IBANs Processed {len(valid_ibans) + len(invalid_ibans)} IBANs.")
    print(f"IBANs Valid: {len(valid_ibans)}")
    print(f"IBANs Invalid: {len(invalid_ibans)}")

if __name__ == "__main__":
    main()

```

Intermediate Python: Exercise #2

Advanced Variables

Complexity level	Solo, Pair or Team?	Target time
		 30-45 minutes

Brief

This exercise presents a problem for you to solve using Python. It should reinforce your understanding of more advanced Python types and help to prepare you for the challenges of this programme.

During the problem-solving process you should feel able to communicate your ideas with your peers if that helps.

When you have completed this task, you should ask your instructor for constructive feedback. Comparing and contrasting your solution with your peers is also highly recommended.

Tools overview

You will be using a variety of software tools:

- **Python 3** (you are recommended to use Python version 3.12)
 - a high-level, general purpose programming language favoured by Data Scientists for experiments
 - <https://www.python.org/>
- **A suitable IDE**, e.g.
 - Microsoft Visual Studio Code
 - PyCharm Community Edition
 - IDLE

Solutions should be tested on a Microsoft Windows platform but should work, largely unaltered on others, e.g. Linux distributions, Apple Mac OS X etc., including through the use of containerisation and virtualisation.

Expected outcome(s)

On success, you should be able to confidently solve a problem by writing appropriately structured and idiomatic Python 3 code which reflects good practice and follows PEP8 guidelines.

Set Problem

You have been asked to create a Python script which will create a sequence of unique **transaction reference numbers**.

As you are no doubt aware, transaction reference numbers are simply unique IDs for payments and may be random or timestamp-based in nature, often featured in Faster Payments, CHAPS payments, Internal ledger postings and audit trails.

Typically, they include:

- A timestamp (for ordering & traceability)
- A sequence number (for uniqueness within the same second)
- An Optional checksum (for integrity)

You will be using the following dash separated format:

PREFIX-YYYYMMDD-HHMMSS-SEQ

Where:

PREFIX	TXN (abbreviation of “transaction”; a default value)
YYYY	Year (4 digits, e.g. 2026)
MM	Month (2 digits, e.g. 07 for July)
DD	Day (2 digits, e.g. 12 for the 12 th day of the month)
HH	Hours (2 digits, 24-hour clock, e.g. 00→12)
MM	Minutes (2 digits, e.g. 00→59)
SS	Seconds (2 digits, e.g. 00→59)
SEQ	A leading zero padded counter of 4 digits , starting at 0001 each second.

You will need to create a single function (“transaction_reference_generator”) which should **yield** successive Transaction references, i.e. a function providing a single value, rather than a complete list.

You will also need some test code to print the first 20 transaction references in the sequence.

Hint:

Given the use of the timestamp in the transaction reference, a short time delay of a few milliseconds may be helpful to distinguish them.

You may use any language features you are currently familiar with or research likely candidates; you may also ask your instructor for suggestions, if you like.

Stretch objectives (Things to try)

4. Ensure that your code is PEP8 compliant by using your IDE or available tooling.
 5. Include suitable error handling that prevents the script from crashing if the file cannot be found/opened (e.g. due to permissions) etc. Correctly report any errors to `stderr`, rather than `stdout`.
 6. Use an appropriate linting tool to discover any potential problems with your code, e.g. unnecessary imports, unused variables etc.
-

Additional notes and reflection points

- Which data structures did you select as part of your solution?
 - Did you find a better solution when comparing your solution with your peers?
 - Which part of the solution did you find the most complex to solve?
 - Did you choose EAFP or LBYL error handling techniques?
-

Further reading:

You can find reference for `strftime()` format codes here:

<https://www.geeksforgeeks.org/python/python-strftime-function/>

<https://docs.python.org/3.12/library/time.html>

Intermediate Python: Sample Solution #2

Advanced Variables

Below is a sample Python script illustrating some of the key steps from the practical. There are multiple ways to approach this task, so your code does not need to match exactly, but it should be clear, well-structured, and PEP 8 compliant. And, hopefully, solve the problem as set.

```
import time
from datetime import datetime, timezone

def transaction_reference_generator(prefix: str = "TXN"):
    """
    A generator that yields unique transaction references.

    Format:
    PREFIX-YYYYMMDD-HHMMSS-SEQ

    The sequence facet will reset each second.
    """
    last_timestamp = None
    seq = 0

    while True:
        now = datetime.now(timezone.utc)
        timestamp = now.strftime("%Y%m%d-%H%M%S")

        if timestamp == last_timestamp:
            seq += 1
        else:
            seq = 1
            last_timestamp = timestamp

        yield f"{prefix}-{timestamp}-{seq:04d}"

# Demo: print first 20 values
gen = transaction_reference_generator()

for _ in range(20):
    print(next(gen))
    # A small delay to show sequencing behaviour (per second)
    time.sleep(0.10)
```

Sample output:

```
TXN-20260409-161222-0001
TXN-20260409-161222-0002
TXN-20260409-161222-0003
TXN-20260409-161222-0004
TXN-20260409-161222-0005
TXN-20260409-161222-0006
TXN-20260409-161222-0007
TXN-20260409-161222-0008
TXN-20260409-161223-0001
```

```
TXN-20260409-161223-0002  
TXN-20260409-161223-0003  
TXN-20260409-161223-0004  
TXN-20260409-161223-0005  
TXN-20260409-161223-0006  
TXN-20260409-161223-0007  
TXN-20260409-161223-0008  
TXN-20260409-161223-0009  
TXN-20260409-161223-0010  
TXN-20260409-161224-0001  
TXN-20260409-161224-0002
```

Intermediate Python: Exercise #3

Advanced Functions

Complexity level	Solo, Pair or Team?	Target time
		 30 minutes

Brief

This exercise presents a problem for you to solve using Python. It is intended to reinforce your understanding of advanced functions and support your preparation for the wider challenges of the programme.

Throughout the process, you are encouraged to share and discuss ideas with your peers, particularly your understanding of the given problem.

After completing the task, don't forget to seek constructive feedback from your instructor.

Comparing your solution with those of your peers is also strongly recommended as different techniques and approaches may come to light which may prove useful to you.

Tools overview

You will be using a variety of software tools:

- **Python 3** (you are recommended to use Python version 3.12)
 - a high-level, general purpose programming language favoured by Data Scientists for experiments
 - <https://www.python.org/>
- **A suitable IDE**, e.g.
 - Microsoft Visual Studio Code
 - PyCharm Community Edition
 - IDLE

Solutions should be tested on a Microsoft Windows platform but should work, largely unaltered on others, e.g. Linux distributions, Apple Mac OS X etc., including through the use of containerisation and virtualisation.

Expected outcome(s)

On success, you should be able to confidently solve a function-related problem by writing appropriately structured and idiomatic Python 3 code which reflects good practice and follows PEP8 guidelines.

Set Problem

You have been asked to create a Python script which will provide a single input validation mechanism for various operations.

You will need to create a decorator function called “`validate_input`”.

The decorator function’s sole responsibility is to ensure that an argument passed to a library of financial services functions is valid – in this case, *that it is a string value*. If this is *not* the case, an appropriate labelled **`TypeError`** should be raised.

The three functions which require decoration are:

- `check_sort_code()`
- `check_uk_account_number()`
- `check_payment_card_number()`

You may either manually “monkey patch” or use `@` notation – the latter is preferred from a currency and best practice perspective.

Hint: Strings may be passed with leading or trailing spaces; these must be stripped before any specific processing occurs. However, they are not part of the decorator’s mandate.

Individually, the three checks for each function are:

<code>check_sort_code()</code>	string must be in NN-NN-NN format, where each N is a digit
<code>check_uk_account_number()</code>	string must be 8 digits, padded with leading zeroes, if needed
<code>check_payment_card_number()</code>	string must be 16 digits, starting with either a 4 or 5

If the passed data validates acceptably, the decorated function should return a `True`. If the function’s specific validation check fails, a suitably labelled `ValueError` should be raised.

You should test your code using a mixture of invalid and valid arguments to check its outcomes.

You may use any language features you are currently familiar with or research likely candidates; you may also ask your instructor for suggestions, if you like.

Stretch objectives (Things to try)

1. Ensure that your code is PEP8 compliant by using your IDE or available tooling.
 2. Include suitable error handling logic that prevents the script from crashing if the file cannot be found/opened (e.g. due to permissions) etc. Correctly report any errors to `stderr`, rather than `stdout`.
 3. Use an appropriate linting tool to discover any potential problems with your code, e.g. unnecessary imports, unused variables etc.
 4. Add additional function decorators for new operations, e.g. `check_gb_iban()`
 5. Create a new “debug” decorator that would show the calling arguments and the type of the return value. Decorate your code with this and re-run the script, observing the output.
-

Reflection points

- Which data structures did you select as part of your solution?
 - Did you find a better solution when comparing your solution with your peers?
 - Which part of the solution did you find the most complex to solve?
 - Did you choose EAFP or LBYL error handling techniques?
-

Intermediate Python: Solution #3

Advanced Functions

Below is a sample Python script illustrating some of the key steps from the practical. There are multiple ways to approach this task, so your code does not need to match exactly, but it should be clear, well-structured, and PEP 8 compliant. And, hopefully, solve the problem as set.

```
from functools import wraps
import re

def validate_input(func):
    @wraps(func)
    def wrapper(data):
        if not isinstance(data, str):
            raise TypeError("Your input must be a string")
        return func(data)
    return wrapper

@validate_input
def check_sort_code(data):
    data = data.strip()
    if re.fullmatch(r"\d{2}-\d{2}-\d{2}", data) is None:
        raise ValueError("Invalid sort code!")
    return True

@validate_input
def check_uk_account_number(data):
    data = data.strip()
    if re.fullmatch(r"\d{8}", data) is None:
        raise ValueError("Invalid UK account number!")
    return True

@validate_input
def check_payment_card_number(data):
    data = data.strip()
    if re.fullmatch(r"[45]\d{15}", data) is None:
        raise ValueError("Invalid payment card number!")
    return True

if __name__ == "__main__":
    # Valid
    print(check_sort_code(" 12-34-56 "))
    print(check_uk_account_number(" 12345678 "))
    print(check_payment_card_number(" 4123456789012345 "))
```

```
# Invalid - different technique
for fn, value in [
    (check_sort_code, 123456),
    (check_sort_code, " 123456 "),
    (check_uk_account_number, " 1234 "),
    (check_payment_card_number, " 6123456789012345 "),
]:
    try:
        print(fn(value))
    except Exception as e:
        print(e)
```

Note. The lack of comments and type hinting is deliberate here.

Sample output:

```
True
True
True
Your input must be a string
Invalid sort code!
Invalid UK account number!
Invalid payment card number!
```

Intermediate Python: Exercise #4

Advanced Typing

Complexity level	Solo, Pair or Team?	Target time
		 20 minutes

Brief

This exercise presents a problem for you to solve using Python. It is intended to reinforce your understanding of type hinting support your preparation for the wider challenges of the programme.

Throughout the process, you are encouraged to share and discuss ideas with your peers, particularly your understanding of the given problem.

After completing the task, don't forget to seek constructive feedback from your instructor.

Comparing your solution with those of your peers is also strongly recommended as different techniques and approaches may come to light which may prove useful to you.

Tools overview

You will be using a variety of software tools:

- **Python 3** (you are recommended to use Python version 3.12)
 - a high-level, general purpose programming language favoured by Data Scientists for experiments
 - <https://www.python.org/>
- **A suitable IDE**, e.g.
 - Microsoft Visual Studio
 - PyCharm Community Edition
 - IDLE
- **mypy**
 - A popular static-type checker for Python
 - <https://www.mypy-lang.org/>

Solutions should be tested on a Microsoft Windows platform but should work, largely unaltered on others, e.g. Linux distributions, Apple Mac OS X etc., including through the use of containerisation and virtualisation.

Expected outcome(s)

On success, you should be able to confidently use advanced typing to improve the documentation of a previously solved function-related problem.

Set Problem

Revisit the solution you created for Exercise 03, which included the creation of the decorator function called “`validate_input`”.

Examine this code and add suitable comments and type hinting to improve its documentation for other Python software engineers who may end up reviewing or reusing the code in the future. Your error messages may also require some improvement.

Ideally, type hinting should cover:

- Each function’s arguments
- Each function’s return types
- Any initialisation of local and global variables

You will also need to add suitable type hints for the following functions:

- `check_sort_code()`
- `check_uk_account_number()`
- `check_payment_card_number()`

You may use any language features you are currently familiar with or research likely candidates; you may also ask your instructor for suggestions, if you like.

It is suggested that you use **mypy**, a popular static-type checker, to check whether your provided type hints, initialisations and usage match appropriately – if not, this could identify potential issues at run-time.

Hints

- Don’t worry: a sample solution is available if you did not complete exercise 03 satisfactorily (see `EG_03_INT_PYTH3_ADVANCED_FUNCTIONS_V1.0.docx`)
- Ideally you should run the **mypy** tool with the `--strict` flag.
This ensures you never get a type-related error at runtime without a corresponding (and predictive) **mypy** error.

Additional notes and reflection points

- Did your **mypy** static-type check pass successfully?
 - If not, which issues did you successfully identify and how did you fix them?
 - Which part of the solution did you find the most difficult to code correctly?
 - Would there have been any changes with earlier versions of Python, e.g. when considering changes to the **Typing** module?
-

Stretch Exercises (More things to try)

1. Ensure code is PEP8 compliant and passes inspection by using `mypy -v`.
 2. Investigate the **mypy** tool:
<https://www.mypy-lang.org/>
 3. Investigate the **MonkeyType** tool:
<https://monkeytype.readthedocs.io/en/stable/generation.html>
-

Advanced Python: Solution #4

Advanced Typing

Below is a sample Python script illustrating some of the key steps from the practical. There are multiple ways to approach this task, so your code does not need to match exactly, but it should be clear, well-structured, and PEP 8 compliant. And, hopefully, solve the problem as set.

```
from __future__ import annotations

import re
from functools import wraps
from typing import Any, Callable, TypeVar, cast

F = TypeVar("F", bound=Callable[[str], bool])

def validate_input(func: F) -> F:
    """
    Ensure the provided input is a string before calling the wrapped function.

    The wrapped function is responsible for domain-specific validation.
    """

    @wraps(func)
    def wrapper(data: object) -> bool:
        if not isinstance(data, str):
            raise TypeError("Your input must be a string")
        return func(data)

    return cast(F, wrapper)

@validate_input
def check_sort_code(data: str) -> bool:
    """
    Validate that a UK sort code matches the format NN-NN-NN.

    Leading/trailing whitespace is ignored.
    """
    data = data.strip()

    if re.fullmatch(r"\d{2}-\d{2}-\d{2}", data) is None:
        raise ValueError(
            "Invalid sort code. Expected format: 2digits-2digits-2digits."
        )
    return True

@validate_input
def check_uk_account_number(data: str) -> bool:
    """
    Validate that a UK account number consists of exactly 8 digits.

    Leading/trailing whitespace is ignored.
    """
    data = data.strip()

    if re.fullmatch(r"\d{8}", data) is None:
```

```

        raise ValueError("Invalid UK account number. Expected exactly 8 digits.")
    return True

@validate_input
def check_payment_card_number(data: str) -> bool:
    """
    Validate that a payment card number is 16 digits long and starts with 4 or 5.

    Leading/trailing whitespace is ignored.
    """
    data = data.strip()

    if re.fullmatch(r"[45]\d{15}", data) is None:
        raise ValueError(
            "Invalid payment card number. Expected 16 digits starting with 4 or 5."
        )
    return True

if __name__ == "__main__":
    examples: list[tuple[str, Callable[[str], bool], str]] = [
        ("Sort code", check_sort_code, " 12-34-56 "),
        ("UK account number", check_uk_account_number, " 12345678 "),
        ("Payment card number", check_payment_card_number, " 4123456789012345 "),
    ]

    for label, function, value in examples:
        print(f"{label}: {function(value)}")

    invalid_examples: list[tuple[Callable[[Any], bool], Any]] = [
        (check_sort_code, 123456),
        (check_sort_code, " 123456 "),
        (check_uk_account_number, " 1234 "),
        (check_payment_card_number, " 6123456789012345 "),
    ]

    for function, value in invalid_examples:
        try:
            function(value)
        except (TypeError, ValueError) as exc:
            print(f"{function.__name__}({value!r}) failed: {exc}")

```

Sample outputs:

```

mypy --strict .\EG04_Solution.py

```

```

Success: no issues found in 1 source file

```

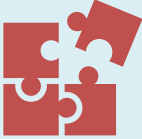
```

Sort code: True
UK account number: True
Payment card number: True
check_sort_code(123456) failed: Your input must be a string
check_sort_code(' 123456 ') failed: Invalid sort code. Expected format: 2digits-2digits-2digits.
check_uk_account_number(' 1234 ') failed: Invalid UK account number. Expected exactly 8 digits.
check_payment_card_number(' 6123456789012345 ') failed: Invalid payment card number. Expected 16 digits starting with 4 or 5.

```

Intermediate Python: Exercise #5

Further Object-Oriented Programming

Complexity level	Solo, Pair or Team?	Target time
		 30 minutes

Brief

This exercise presents a problem for you to solve using Python. It should reinforce your Python fundamentals and the more nuanced features of object-oriented programming.

Throughout the process, you are encouraged to share and discuss ideas with your peers, particularly your understanding of the given problem.

After completing the task, don't forget to seek constructive feedback from your instructor.

Comparing your solution with those of your peers is also strongly recommended as different techniques and approaches may come to light which may prove useful to you.

Tools overview

You will be using a variety of software tools:

- **Python 3** (you are recommended to use Python version 3.12)
 - a high-level, general purpose programming language favoured by Data Scientists for experiments
 - <https://www.python.org/>
- **A suitable IDE**, e.g.
 - Microsoft Visual Studio Code
 - PyCharm Community Edition
 - IDLE

Solutions should be tested on a Microsoft Windows platform but should work, largely unaltered on others, e.g. Linux distributions, Apple Mac OS X etc., including through the use of containerisation and virtualisation.

Expected outcome(s)

On success, you should be able to confidently use object-oriented concepts to design and implement a suitable custom iterator using selected magic methods.

Set Problem

If you recall, previously in Exercise #02, you were asked to create a Python script which created a sequence of unique **transaction reference numbers**.

Typically, such transaction reference numbers include faceted data as:

- A timestamp (for ordering & traceability)
- A sequence number (for uniqueness within the same second)
- An Optional checksum (for integrity)

You used the following dash separated format:

PREFIX-YYYYMMDD-HHMMSS-SEQ

Where:

PREFIX	TXN (abbreviation of “transaction”; a default value)
YYYY	Year (4 digits, e.g. 2026)
MM	Month (2 digits, e.g. 07 for July)
DD	Day (2 digits, e.g. 12 for the 12 th day of the month)
HH	Hours (2 digits, 24-hour clock, e.g. 00→12)
MM	Minutes (2 digits, e.g. 00→59)
SS	Seconds (2 digits, e.g. 00→59)
SEQ	A leading zero padded counter of 4 digits, starting at 0001 each second.

Initially you wrote a single function (“transaction_reference_generator”) which **yielded** successive Transaction references, i.e. a function providing a single value, rather than a complete list, using a *stateful generator object*.

In Exercise #05 we’d like you to implement it as a **custom iterator object**, *rather* than a generator. Call this class “**TransasctionReferenceIterator**”.

You will need to code (at least) three key magic methods in this class, i.e.

1. `__init__()`
2. `__iter__()`
3. `__next__()`

Test your iterator object with the following test runner sample code:

```
def test_transaction_reference_iterator():
    iterator = TransactionReferenceIterator()
    # Print the first 20 transaction numbers
    for _ in range(20):
        print(next(iterator))
        time.sleep(0.10)

# Run our test sequence
if __name__ == "__main__":
```

```
test_transaction_reference_iterator()
```

In addition, you may use any language features you are currently familiar with or research likely candidates; you may also ask your instructor for suggestions, if you like.

Hint

Don't forget that this isn't a generator – you will need to *manually maintain* the iterator's state within the object...

Stretch objectives (Things to try)

6. Ensure your code is PEP8 compliant by using available tools.
 7. Try to use a linter to discover potential problems with your code, e.g. unnecessary imports, unused variables etc.
 8. Add suitable code within your class to override the implementation for **default string** and **representation** magic methods.
-

Additional notes and reflection points

- Which data structures did you select as part of your solution?
 - Did you find a better solution when comparing your solution with your peers?
 - Which part of the solution did you find the most complex to solve?
 - Did you choose EAFP or LBYL error handling techniques?
 - Why would we not just “**for**” loop over the iterator itself?
 - Which implementation do you prefer: generator or iterator? - which is best (and why)?
 - Do you need to raise a **StopIterator** exception? – if not, why?
-

Intermediate Python: Solution #5

Further Object-Oriented Programming

The following is a sample script that covers some of the basic steps of the above practical. There are many possible ways to write code like this, so yours may not look exactly the same but it should be PEP8-compliant and readable.

```
import time
from datetime import datetime, timezone

class TransactionReferenceIterator:
    """
    A Customer Iterator that yields unique transaction references.

    Format:
    PREFIX-YYYYMMDD-HHMMSS-SEQ

    Sequence resets each second.
    """

    def __init__(self, prefix: str = "TXN"):
        self.prefix = prefix
        self.last_timestamp = None
        self.seq = 0

    def __iter__(self):
        return self

    def __next__(self):
        now = datetime.now(timezone.utc)
        timestamp = now.strftime("%Y%m%d-%H%M%S")
        if timestamp == self.last_timestamp:
            self.seq += 1
        else:
            self.seq = 1
            self.last_timestamp = timestamp

        return f"{self.prefix}-{timestamp}-{self.seq:04d}"

def test_transaction_reference_iterator():
    iterator = TransactionReferenceIterator()
    # Print the first 20 transaction numbers
    for _ in range(20):
        print(next(iterator))
        time.sleep(0.10)

# Run our test sequence
if __name__ == "__main__":
    test_transaction_reference_iterator()
```

Sample Output

```
TXN-20260411-154650-0001
TXN-20260411-154651-0001
TXN-20260411-154651-0002
TXN-20260411-154651-0003
TXN-20260411-154651-0004
TXN-20260411-154651-0005
TXN-20260411-154651-0006
TXN-20260411-154651-0007
TXN-20260411-154651-0008
TXN-20260411-154651-0009
TXN-20260411-154651-0010
TXN-20260411-154652-0001
TXN-20260411-154652-0002
TXN-20260411-154652-0003
TXN-20260411-154652-0004
TXN-20260411-154652-0005
TXN-20260411-154652-0006
TXN-20260411-154652-0007
TXN-20260411-154652-0008
TXN-20260411-154652-0009
```

Intermediate Python: Exercise #6

Introspection

Complexity level	Solo, Pair or Team?	Target time
		 30 minutes

Brief

This exercise presents a problem for you to solve using Python. It is intended to reinforce your understanding of introspection concepts and support your preparation for the wider challenges of the programme.

Throughout the process, you are encouraged to share and discuss ideas with your peers, particularly your understanding of the given problem.

After completing the task, don't forget to seek constructive feedback from your instructor.

Comparing your solution with those of your peers is also strongly recommended as different techniques and approaches may come to light which may prove useful to you.

Tools overview

You will be using a variety of software tools:

- **Python 3** (you are recommended to use Python version 3.12)
 - a high-level, general purpose programming language favoured by Data Scientists for experiments
 - <https://www.python.org/>
- **A suitable IDE**, e.g.
 - Microsoft Visual Studio Code
 - PyCharm Community Edition
 - IDLE

Solutions should be tested on a Microsoft Windows platform but should work, largely unaltered on others, e.g. Linux distributions, Apple Mac OS X etc., including through the use of containerisation and virtualisation.

Expected outcome(s)

On success, you should be able to confidently use object-oriented concepts and introspection to design and implement a class-based validation system.

Set Problem

Create a class-based validation system for a customer's profile.

The **CustomerProfileValidation** class should (minimally) contain the following properties and methods:

Class UserValidator
Properties
- email - mobile - customerID
Methods
- validate_email() - validate_mobile() - validate_customerID() + run_validations()

The three required validation rules are:

1. email – a recognisable corporate-style format, e.g. [frank.lastname@domain.com](mailto:frank.boifur@qa.com)
2. mobile – 10 digits
3. customerID – 8 digits

Use the following boilerplate code to test your solution:

```
# Testing your validator
def test_customer_profile_validator():
    customer = CustomerProfileValidator(
        email="frank.boifur@qa.com",
        mobile="071 234 5678",
        customerID="01234567",
    )
    results = customer.run_validations()
    return results

# Run your test
if __name__ == "__main__":
    print(test_customer_profile_validator())
```

In addition, you may use any language features you are currently familiar with or research likely candidates; you may also ask your instructor for suggestions, if you like.

Hints

- You have free reign how to technically achieve these validation tasks.
 - The **run-validations** method should introspect the validation methods dynamically, execute them and return a single Boolean, i.e. whether the properties ALL pass validation.
-

Stretch objectives (Things to try)

9. Ensure that your code is PEP8 compliant by using your IDE or available tooling.
10. Include suitable error that prevents the script from crashing if the file cannot be found/opened (e.g. due to permissions) etc. Correctly report any errors to `stderr`, rather than `stdout`.
11. Use an appropriate linting tool to discover any potential problems with your code, e.g. unnecessary imports, unused variables etc.
12. Include a more detailed mechanism which reports exactly which validations have failed.
13. Add an “age” property for the customer and validate this between 18 and 67 (inclusive) – if added and coded correctly, it should just work...

Additional notes and reflection points

- Which data structures did you select as part of your solution?
 - Did you find a better solution when comparing your solution with your peers?
 - Which part of the solution did you find the most complex to solve?
 - Did you choose EAFP or LBYL error handling techniques?
 - Did you find introspection a useful feature of Python? Had you used it before?
-

Intermediate Python: Solution #6

Introspection

The following is a sample script that covers some of the basic steps of the above practical. There are many possible ways to write code like this, so yours may not look exactly the same but it should be PEP8-compliant and readable.

Note. This version also includes the solution to Stretch task #4.

```
import re

class CustomerProfileValidator:
    def __init__(self, email: str, mobile: str, customerID: str):
        self.email = email
        self.mobile = mobile
        self.customerID = customerID

    def _validate_email(self) -> bool:
        """Validate email"""
        email = self.email.strip().lower()
        return re.fullmatch(r"^[a-z]+\.[a-z]+@[a-z]+\.[a-z]{2,}$", email) is not None

    def _validate_mobile(self) -> bool:
        """Validate mobile telephone"""
        mobile = self.mobile.replace(" ", "")
        return re.fullmatch(r"\d{10}", mobile) is not None

    def _validate_customer_id(self) -> bool:
        """Validate Customer ID"""
        customerID = self.customerID.strip()
        return customerID.isdigit() and len(customerID) == 8

    def run_validations(self) -> dict[str, bool]:
        """Identify and run all validation checks in this class"""
        results = {
            method.__doc__: method()
            for name in dir(self)
            if name.startswith("_validate_")
            and callable(method := getattr(self, name))
            and callable(getattr(type(self), name, None))
        }
        results["all_valid"] = all(results.values())
        return results

# Testing your validator
def test_customer_profile_validator():
```

```
customer = CustomerProfileValidator(  
    email="frank.boifur@qa.com",  
    mobile="071 234 5678",  
    customerID="01234567",  
)  
results = customer.run_validations()  
return results  
  
# Run your test  
if __name__ == "__main__":  
    print(test_customer_profile_validator())
```

Sample Output:

```
{'Validate Customer ID': True, 'Validate email': True, 'Validate mobile telephone':  
True, 'all_valid': True}
```

Intermediate Python: Exercise #7

Deeper Object-Oriented

Complexity level	Solo, Pair or Team?	Target time
		 30 minutes

This exercise presents a problem for you to solve using Python. It is intended to reinforce your understanding of more complex object-oriented programming concepts and support your preparation for the wider challenges of the programme.

Throughout the process, you are encouraged to share and discuss ideas with your peers, particularly your understanding of the given problem.

After completing the task, don't forget to seek constructive feedback from your instructor.

Comparing your solution with those of your peers is also strongly recommended as different techniques and approaches may come to light which may prove useful to you.

Tools overview

You will be using a variety of software tools:

- **Python 3** (you are recommended to use Python version 3.12)
 - a high-level, general purpose programming language favoured by Data Scientists for experiments
 - <https://www.python.org/>
- **A suitable IDE**, e.g.
 - Microsoft Visual Studio Code
 - PyCharm Community Edition
 - IDLE

Solutions should be tested on a Microsoft Windows platform but should work, largely unaltered on others, e.g. Linux distributions, Apple Mac OS X etc., including through the use of containerisation and virtualisation.

Expected outcome(s)

On success, you should be able to implement a popular design pattern, in this case a behavioural one, to solve a real-world problem related to a common industry service.



Sort code primer

Sort codes are not exclusively owned forever — over time, banks merge and migrate.

For example, some Lloyds sort codes may now route via:

- Halifax (part of the same group)
- Bank of Scotland (all under Lloyds Banking Group)

The authoritative source for sort codes is the Extended Industry Sort Code Directory (**EISCD**) database. Essentially, the new version of the Industry Sort Code Directory (**ISCD**).

This EISCD database also includes critical information such as the:

- Bank office identifier (BIC)
- Branch title
- Clearing that the bank office participates in
- Postal address
- Telephone number

Due to its ever-changing nature, lookups such as this shouldn't be hardcoded in a solution. Moreover, it's useful to know when there have been changes...

In this problem, we'll look at an object-oriented technique that allow us to know when (and what) changes have been made to a synthetic **EISCD**, using a preferred communications channel.

Set Problem

Create a class-based notification system for the EISCD.

A team uses EISCD to look up sort codes for bank branches referenced within their business unit. Changes (including additions and removals) to the database are flagged using a digest system. This can either be sent to its registered users via SMS or email.

Important changes logged include updated, closed and redirected sort codes.

Upon joining the business unit, users are asked to provide their communication details and state their preferred communication channel. As noted, this currently supports only email or SMS.

When changes are made to our synthetic EISCD database, a suitable message should be automatically sent to each registered user's preferred channel – unless the user has requested no active contact, which is a viable option.

Create suitable class schemas and Python code to implement and test this system using the following boilerplate code (and sample test data) provided, e.g.

```
# Test
def main() -> None:
    notifier = EISCDChangeNotifier()

    # Register 3 different users: one email, one mobile, one none
    email_user = User(
        name="Frank",
        subscription_type="email",
        destination="frank.boifur@qa.com",
    )
    mobile_user = User(
        name="Jess",
        subscription_type="mobile",
        destination="+44712900123",
    )
    none_user = User(
        name="Phil",
        subscription_type="none",
        destination=None,
    )

    notifier.subscribe(email_user, authorize_email)
    notifier.subscribe(mobile_user, authorize_mobile)
    notifier.subscribe(none_user, authorize_none)

    # First notification
    notifier.notify(
        "EISCD weekly refresh has been loaded: 13 sort codes updated, 2 closed, 1
redirected."
    )

    # Remove the mobile user from the business unit
    notifier.unsubscribe(mobile_user)

    # Second notification
    notifier.notify(
        "EISCD weekly refresh has been loaded: 7 sort codes updated, 1 CHAPS flag
changed."
    )

if __name__ == "__main__":
    main()
```

This should output, e.g.

```
[SUBSCRIBE] Frank registered with type='email'
[SUBSCRIBE] Jess registered with type='mobile'
[SUBSCRIBE] Phil registered with type='none'

=====
[PUBLISH] Weekly EISCD update: EISCD weekly refresh has been loaded: 13 sort codes
updated, 2 closed, 1 redirected.
=====
[CALLBACK] Triggering callback for Frank (email)
[EMAIL AUTH] Email notification authorized for message: EISCD weekly refresh has been
loaded: 13 sort codes updated, 2 closed, 1 redirected.
[CALLBACK] Triggering callback for Jess (mobile)
[MOBILE AUTH] Mobile notification authorized for message: EISCD weekly refresh has
been loaded: 13 sort codes updated, 2 closed, 1 redirected.
[CALLBACK] Triggering callback for Phil (none)
[NO AUTH] User has opted out of all communication channels. No notification sent for
message: EISCD weekly refresh has been loaded: 13 sort codes updated, 2 closed, 1
redirected.
[UNSUBSCRIBE] Jess removed from notifications

=====
[PUBLISH] Weekly EISCD update: EISCD weekly refresh has been loaded: 7 sort codes
updated, 1 CHAPS flag changed.
=====
[CALLBACK] Triggering callback for Frank (email)
[EMAIL AUTH] Email notification authorized for message: EISCD weekly refresh has been
loaded: 7 sort codes updated, 1 CHAPS flag changed.
[CALLBACK] Triggering callback for Phil (none)
[NO AUTH] User has opted out of all communication channels. No notification sent for
message: EISCD weekly refresh has been loaded: 7 sort codes updated, 1 CHAPS flag
changed.
```

Hint:

You may find a Python **dataclass** useful, but it is not mandatory! Note: The `@dataclass` decorator that comes from the built-in `dataclasses` module is a convenient way to automatically generate common methods for classes that mainly store data. It will automatically create `__init__`, `__repr__`, `__eq__` (and optionally `__it__`, etc.) methods for you.

In addition, you may use any language features you are currently familiar with or research likely candidates; you may also ask your instructor for suggestions, if you like.

Stretch objectives (Things to try)

14. Ensure that your code is PEP8 compliant by using your IDE or available tooling.
15. Include suitable error that prevents the script from crashing if run-time issues occur etc. Correctly report any errors to `stderr`, rather than `stdout`.
16. Use an appropriate linting tool to discover any potential problems with your code, e.g. unnecessary imports, unused variables etc.

Additional notes and reflection points

- Which data structures did you select as part of your solution?
- Did you find a better solution when comparing your solution with your peers?

- Which part of the solution did you find the most complex to solve?
 - Did you choose EAFP or LBYL error handling techniques?
 - Did you find introspection a useful feature of Python? Had you used it before?
-

Intermediate Python: Solution #7

Deeper Object-Oriented

The following is a sample script that covers some of the basic steps of the above practical. There are many possible ways to write code like this, so yours may not look exactly the same but it should be PEP8-compliant and readable.

```
from __future__ import annotations

from dataclasses import dataclass, field
from typing import Callable

# Subscriber model
AuthCallback = Callable[[str], None]

@dataclass(eq=True, frozen=True)
class User:
    name: str
    subscription_type: str # "email", "mobile", or "none"
    destination: str | None = None

# Notification authorization / delivery callback methods
def authorize_email(message: str) -> None:
    print(f"[EMAIL AUTH] Email notification authorized for message: {message}")

def authorize_mobile(message: str) -> None:
    print(f"[MOBILE AUTH] Mobile notification authorized for message: {message}")

def authorize_none(message: str) -> None:
    print(f"[NO AUTH] User has opted out of all communication channels. No notification sent for message: {message}")

# Publisher
@dataclass
class EISCDChangeNotifier:
    """
    Publisher in a simple subscriber design pattern.

    Each subscriber is associated with a callback that is executed
    automatically when a weekly EISCD change notification is published.
    """
    _subscribers: dict[User, AuthCallback] = field(default_factory=dict)

    def subscribe(self, user: User, callback: AuthCallback) -> None:
        self._subscribers[user] = callback
        print(
```

```

        f"[SUBSCRIBE] {user.name} registered with
type={user.subscription_type!r}"
    )

    def unsubscribe(self, user: User) -> None:
        removed = self._subscribers.pop(user, None)
        if removed is not None:
            print(f"[UNSUBSCRIBE] {user.name} removed from notifications")
        else:
            print(f"[UNSUBSCRIBE] {user.name} was not subscribed")

    def notify(self, message: str) -> None:
        print("\n" + "=" * 80)
        print(f"[PUBLISH] Weekly EISCD update: {message}")
        print("=" * 80)

        if not self._subscribers:
            print("[PUBLISH] No active subscribers")
            return

        for user, callback in self._subscribers.items():
            print(
                f"[CALLBACK] Triggering callback for {user.name} "
                f"({user.subscription_type})"
            )
            callback(message)

# Test
def main() -> None:
    notifier = EISCDChangeNotifier()

    # Register 3 different users: one email, one mobile, one no channel
    email_user = User(
        name="Frank",
        subscription_type="email",
        destination="frank.boifur@qa.com",
    )
    mobile_user = User(
        name="Jess",
        subscription_type="mobile",
        destination="+44712900123",
    )
    none_user = User(
        name="Phil",
        subscription_type="none",
        destination=None,
    )

    notifier.subscribe(email_user, authorize_email)
    notifier.subscribe(mobile_user, authorize_mobile)
    notifier.subscribe(none_user, authorize_none)

    # First notification
    notifier.notify(
        "EISCD weekly refresh has been loaded: 13 sort codes updated, 2 closed, 1
redirected."
    )

    # Remove the mobile user
    notifier.unsubscribe(mobile_user)

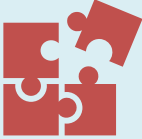
    # Second notification

```

```
    notifier.notify(  
        "EISCD weekly refresh has been loaded: 7 sort codes updated, 1 CHAPS flag  
changed."  
    )  
  
if __name__ == "__main__":  
    main()
```

Intermediate Python: Exercise #8

External Data

Complexity level	Solo, Pair or Team?	Target time
		 30 minutes

Brief

This exercise presents a problem for you to solve using Python. It should reinforce your Python fundamentals and ability to request, parse, and extract data from an external source, e.g. a web-based API.

Throughout the process, you are encouraged to share and discuss ideas with your peers, particularly your understanding of the given problem.

After completing the task, don't forget to seek constructive feedback from your instructor.

Comparing your solution with those of your peers is also strongly recommended as different techniques and approaches may come to light which may prove useful to you.

Tools overview

You will be using a variety of software tools:

- **Python 3** (you are recommended to use Python version 3.12)
 - a high-level, general purpose programming language favoured by Data Scientists for experiments
 - <https://www.python.org/>
- **A suitable IDE**, e.g.
 - Microsoft Visual Studio
 - PyCharm Community Edition
 - IDLE
- **Internet access to** <https://postcodes.io>
 - A free Postcode and Geolocation API for the UK

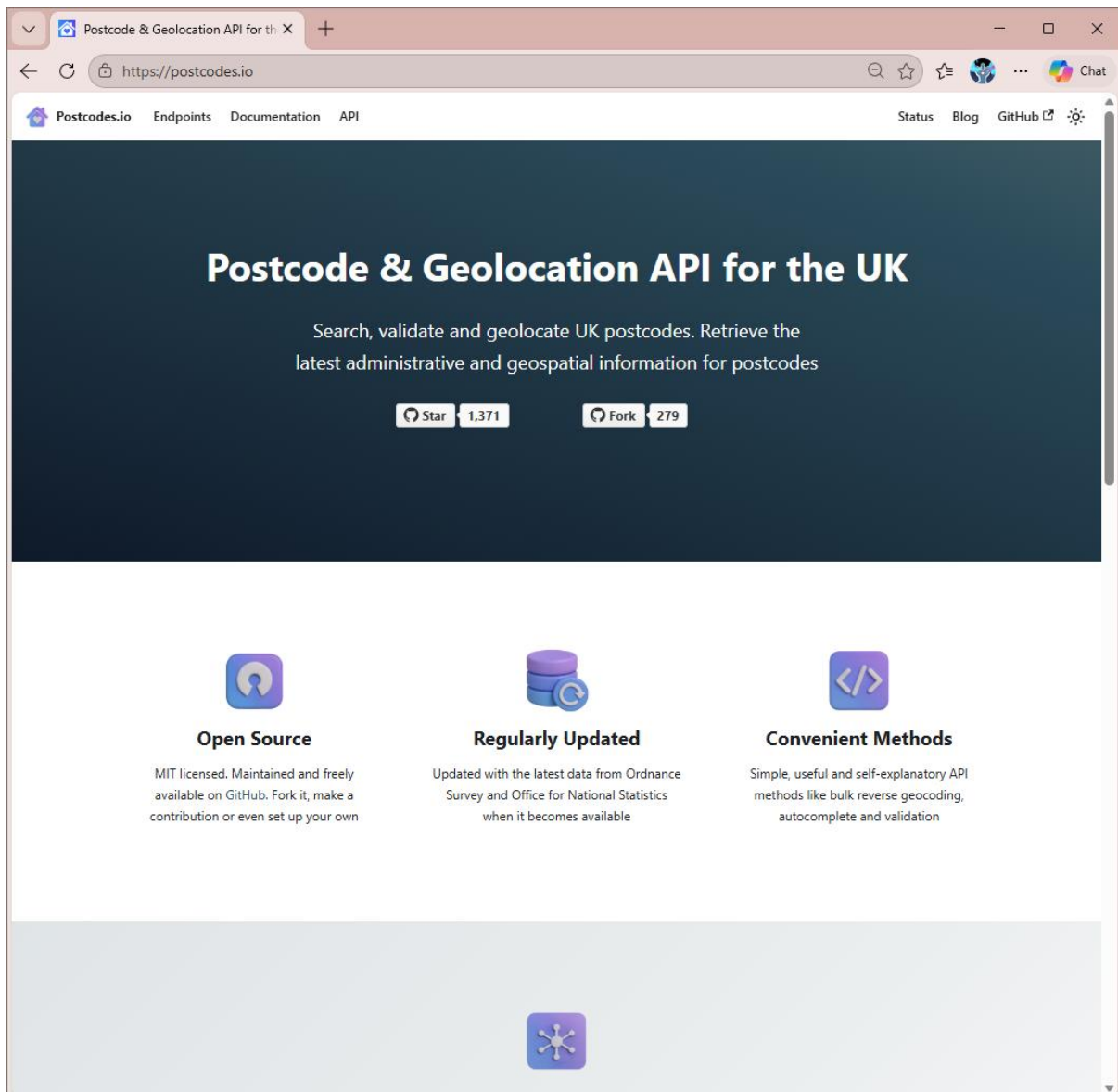
Solutions should be tested on a Microsoft Windows platform but should work, largely unaltered on others, e.g. Linux distributions, Apple Mac OS X etc., including through the use of containerisation and virtualisation.

Expected outcome(s)

On successful completion, you should be able to implement a simple HTTP request to an external data source (in this case a publicly available, free, and key-less API), correctly consume its JSON response and extract the requested information.

Set Problem

Explore the following web-based API: <https://postcodes.io>



You will then:

1. **Identify** the **correct endpoint** and **HTTP(S) method** necessary to lookup a *single postcode*, e.g. **EC2R 8AH**
2. **Write a PEP8-compliant Python script** to create a request using the correct HTTP(S) method and test it with both a valid and invalid postcode.
3. **Write handlers** for both successful and unsuccessful responses, using appropriate HTTP status codes, e.g. for success and failure.
4. **Parse the JSON response and Extract the required information** (*itemised below*), **printing it** onscreen in a suitable format.

Field values required are:

- **country**
 - **parliamentary_constituency**
 - **longitude**
 - **latitude** and
 - **NUTS geocode** (Nomenclature of Territorial Units for Statistics – from the French *Nomenclature des unités territoriales statistiques*)
-

Hints

- You *will* need the **requests** module installed (or an equivalent, e.g. `httpx`)
 - You *may* need to **spoof the user agent string** if the programmatic client request is blocked by the web server – this can sometimes happen when not accessing via a known web browser.
-

Stretch objectives (Things to try)

17. Ensure your code is PEP8 compliant by using available tools.
 18. Try to use a linter to discover potential problems with your code, e.g. unnecessary imports, unused variables etc.
-

Intermediate Python: Solution #8

External Data

The following is a sample script that covers some of the basic steps of the above practical. There are many possible ways to write code like this, so yours may not look exactly the same but it should be pep8-compliant and readable.

```
"""A script demonstrating access to external data."""

from __future__ import annotations

import sys
from typing import Any, Dict, Final, Optional

import requests

POSTCODES_API_BASE_URL: Final[str] = "https://api.postcodes.io/postcodes"
REQUEST_TIMEOUT_SECONDS: Final[int] = 5
SEPARATOR_WIDTH: Final[int] = 60

# Custom headers to spoof the User-Agent
# (in case of API restriction)
REQUEST_HEADERS: Final[Dict[str, str]] = {
    "User-Agent": (
        "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
        "AppleWebKit/537.36 (KHTML, like Gecko) "
        "Chrome/115.0.1901.183 Safari/537.36 Edg/115.0.1901.183"
    )
}

def build_postcode_url(postcode: str) -> str:
    """
    Build the Postcodes.io URL for a given postcode.

    :param postcode: The postcode to lookup
    :return: Fully qualified API URL
    """
    return f"{POSTCODES_API_BASE_URL}/{postcode}"

def fetch_postcode_response(postcode: str) -> requests.Response:
    """
    Make a GET request for a postcode lookup.

    :param postcode: The postcode to lookup
    :return: The HTTP response object
    :raises requests.exceptions.RequestException:
        If the request fails
    """
    url = build_postcode_url(postcode)

    # Make Get request to endpoint URL, with spoofed
    # user agent string and a suitable timeout
    return requests.get(
        url,
        headers=REQUEST_HEADERS,
        timeout=REQUEST_TIMEOUT_SECONDS,
    )
```

```

def extract_postcode_information(data: Dict[str, Any]) -> Dict[str, Any]:
    """
    Extract specific information from the API response payload.

    :param data: Parsed JSON response
    :return: Dictionary containing selected postcode fields
    """
    result: Dict[str, Any] = data.get("result", {})

    extracted_info: Dict[str, Any] = {
        "country": result.get("country"),
        "parliamentary_constituency": result.get("parliamentary_constituency"),
        "longitude": result.get("longitude"),
        "latitude": result.get("latitude"),
        "NUTS_geocode": result.get("codes", {}).get("nuts"),
    }
    return extracted_info

def print_error(message: str) -> None:
    """
    Print an error message to standard error.
    """
    print(message, file=sys.stderr)

def lookup_postcode(postcode: str) -> Optional[Dict[str, Any]]:
    """
    Looks up a single postcode using the Postcodes.io API
    and extract specific information.
    """
    try:
        response = fetch_postcode_response(postcode)

        if response.status_code == requests.codes.ok: # 200
            data: Dict[str, Any] = response.json()
            return extract_postcode_information(data)

        if response.status_code == requests.codes.not_found: # 404
            print_error(f"Error: Postcode '{postcode}' was not found.")
            return None

        print_error(
            f"Error: Received unexpected status code {response.status_code}."
        )
        return None

    except requests.exceptions.RequestException as exc:
        print_error(f"Error: Request failed: {exc}")
        return None

def print_separator() -> None:
    """Display a row of separator characters."""
    print("\n" + "-" * SEPARATOR_WIDTH + "\n")

def print_extracted_information(result: Dict[str, Any]) -> None:
    """Print extracted postcode information."""
    print("Extracted Information:")
    for key, value in result.items():
        print(f"  {key}: {value}")

def main() -> None:
    """Entry point for the script."""

```

```
postcode = "EC2R8AH"

print_separator()
print(f"Testing with postcode: {postcode}")

result = lookup_postcode(postcode)

if result:
    print_extracted_information(result)

print_separator()

if __name__ == "__main__":
    main()
```

Sample Output (e.g. OK 200)

```
-----
Testing with postcode: EC2R8AH
Extracted Information:
  country: England
parliamentary_constituency: Cities of London and Westminster
longitude: -0.088622
latitude: 51.514188
NUTS_geocode: TLI35
-----
```

Intermediate Python: Exercise #9

Networking with Sockets

Complexity level	Solo, Pair or Team?	Target time
	 or	 30 minutes

Brief

This exercise presents a problem for you to solve using Python. It is intended to reinforce your understanding of socket-based networking concepts and support your preparation for the wider challenges of the programme.

Throughout the process, you are encouraged to share and discuss ideas with your peers, particularly your understanding of the given problem.

After completing the task, don't forget to seek constructive feedback from your instructor.

Comparing your solution with those of your peers is also strongly recommended as different techniques and approaches may come to light which may prove useful to you.

Tools overview

You will be using a variety of software tools:

- **Python 3** (you are recommended to use Python version 3.12)
 - a high-level, general purpose programming language favoured by Data Scientists for experiments
 - <https://www.python.org/>
- **A suitable IDE**, e.g.
 - Microsoft Visual Studio Code
 - PyCharm Community Edition
 - IDLE

Solutions should be tested on a Microsoft Windows platform but should work, largely unaltered on others, e.g. Linux distributions, Apple Mac OS X etc., including through the use of containerisation and virtualisation.

Expected outcome(s)

On success, you should be able to confidently create a network client which sends an ISO 8583-style message to a server, which validates it, and transmits back an appropriate response.

Set Problem

You are going to develop a pair of networking scripts which simulate a very stripped-down version of ISO 8583 – the international standard for financial transaction card originated interchange messaging.

This will simulate the typical transmission sent from a shop (i.e. an EFTPOS) device or ATM to a backend server for authorization, typically based on amount and/or available balance.

This type of problem uses many abbreviations and acronyms. Here's a list you may use as reference:

Abbreviation/Acronym	Meaning	Context
ISO 8583	Financial messaging standard	Card payments
EFTPOS	– Electronic Funds Transfer Point of Sale, e.g. Card Terminal	Retail
TCP	Network protocol	Sockets
MTI	Message type	Request/response
PAN	Primary Account Number/Payment Card number	Payment data
AMOUNT	Transaction value	Payment
CUR	Currency code	ISO 4217
RC	Response code	Approval/decline
MSG	Human-readable message	Debug/output
ATM	Automatic Teller Machine (Cash machine)	Banking

You are required to author a short Python script to create network **server code** which first listens then accepts a TCP-based connection from a client, receives a message (in an agreed ISO 8583-style protocol), then authorizes it (or not) and sends back an appropriately coded response.

To help you get started, here's the full **Python code for the client**:

```
from __future__ import annotations
import socket

HOST = "127.0.0.1"
PORT = 5000

def build_auth_request(pan: str, amount: int, currency: str) -> str:
    """
    Build a simple authorization request message.

    amount is in minor units, e.g. pence:
    950 = 9.50 GBP

    MTI 0100 = Authorisation Request ("can I charge?")
    """
    return f"MTI=0100|PAN={pan}|AMOUNT={amount}|CUR={currency}"
```

```
def send_request(message: str) -> str:
    """
    Send a sample request to the server and return the response
    received.
    """
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
        sock.connect((HOST, PORT))
        sock.sendall(message.encode("utf-8"))

        response = sock.recv(4096)

    return response.decode("utf-8")

def main() -> None:
    """
    Simulate a request, send it and print response.
    """
    request = build_auth_request(
        pan="4111111111111111",      # Simulate card
        amount=10075,
        currency="GBP",
    )

    print(f"Sending request: {request}")
    response = send_request(request)
    print(f"Received response: {response}")

if __name__ == "__main__":
    main()
```

You will need to create the **server part** of the solution that this client interacts with.

It will be required to:

- Create and bind an IPv4 TCP connection on port 5000
- Listen for a connection from a client
- Accept the connection
- Receive and parse the message into its constituent parts/fields
- (Optional) Mask all but last 4 digits of the PAN
- Determine the appropriate response, i.e. authorized/not authorized (e.g. based on a transaction limit of £100)
- Encode the response and send the result back to the server
- Safely close the connection

Information that will help

If the client sends:

MTI=0100 | PAN=4111111111111111 | AMOUNT=1250 | CUR=GBP

This means:

MTI = 0100 An authorization REQUEST ("Can I?")

PAN = The customer's card number (12 to 19 digits typically)

AMOUNT = transaction amount (in pence here)

CUR = A 3-letter abbreviation, e.g. GBP (GB Pounds Sterling); could be USD, EUR etc.

Note that the transmission is pipe (“|”) separated.

The server response could be:

```
MTI=0110 | RC=00 | MSG=APPROVED
```

(This means Response Code = 00 approved; the MSG field is designed to be *human-readable*)

Or:

```
MTI=0110 | RC=05 | MSG=DECLINED
```

(This means , 0110 Authorisation RESPONSE (yes/no), Response Code = 05 declined; a 30 would indicate a format error in the request, e.g. insufficient field quantity or incorrectly formatted/length values)

Hints

If necessary, please review the tutorial content demonstrated in DG09 to help.

Stretch objectives (Things to try)

- Ensure that your code is PEP8 compliant by using your IDE or available tooling.
- Include suitable error that prevents the script from crashing if run-time issues occur etc. Correctly report any errors to `stderr`, rather than `stdout`.
- Use an appropriate linting tool to discover any potential problems with your code, e.g. unnecessary imports, unused variables etc.
- Time permitting, you may wrap the socket in SSL/TLS to make the end-to-end data transfer secure.
- Confirm the security applied is working correctly using a suitable tool to capture TCP packets, e.g. wireshark, tcpdump etc.

Additional notes and reflection points

- Which data structures did you select as part of your solution?
- Did you find a better solution when comparing your solution with your peers?
- Which part of the solution did you find the most complex to solve?
- Did you choose EAFP or LBYL error handling techniques?

Intermediate Python: Solution #9

Networking with Sockets

The following is a sample script that covers some of the basic steps of the above practical. There are many possible ways to write code like this, so yours may not look exactly the same but it should be PEP8-compliant and readable.

Server script

```
from __future__ import annotations
import socket
from typing import Dict, Tuple

HOST = "127.0.0.1"
PORT = 5000

def parse_message(message: str) -> Dict[str, str]:
    fields: Dict[str, str] = {}

    for part in message.strip().split("|"):
        if "=" not in part:
            raise ValueError(f"Invalid field format: {part!r}")

        key, value = part.split("=", 1)
        key = key.strip().upper()
        value = value.strip()

        if not key or not value:
            raise ValueError("Empty key or value is not allowed")

        fields[key] = value

    return fields

def mask_pan(pan: str) -> str:
    if len(pan) <= 4:
        return "*" * len(pan)
    return "*" * (len(pan) - 4) + pan[-4:]

def validate_request(fields: Dict[str, str]) -> None:
    required = {"MTI", "PAN", "AMOUNT", "CUR"}
    missing = required - set(fields.keys())
    if missing:
        raise ValueError(f"Missing required fields: {sorted(missing)}")

    if fields["MTI"] != "0100":
        raise ValueError("Only MTI=0100 authorization requests are supported")

    pan = fields["PAN"]
    if not pan.isdigit() or not (12 <= len(pan) <= 19):
        raise ValueError("PAN must be between 12 to 19 digits")

    amount = fields["AMOUNT"]
    if not amount.isdigit():
        raise ValueError("AMOUNT must be an integer in minor units")
```

```

    currency = fields["CUR"]
    if len(currency) != 3 or not currency.isalpha():
        raise ValueError("CUR must be a 3-letter currency code")

def build_response(response_code: str, message: str) -> str:
    return f"MTI=0110|RC={response_code}|MSG={message}"

def process_request(raw_message: str) -> str:
    fields = parse_message(raw_message)
    validate_request(fields)

    amount = int(fields["AMOUNT"])
    masked_pan = mask_pan(fields["PAN"])

    print(
        "Received transaction:",
        f"PAN={masked_pan}",
        f"AMOUNT={amount}",
        f"CUR={fields['CUR']}",
    )
    # limit £100
    if amount < 10_000:
        return build_response("00", "APPROVED")

    return build_response("05", "DECLINED")

def handle_client(conn: socket.socket, addr: Tuple[str, int]) -> None:
    print(f"Connected by {addr}")

    try:
        data = conn.recv(4096)
        if not data:
            print("No data received")
            return

        request = data.decode("utf-8")
        print(f"Raw request: {request}")

        try:
            response = process_request(request)
        except ValueError as exc:
            response = build_response("30", f"FORMAT_ERROR:{exc}")
        except Exception as exc:
            print(f"Unexpected processing error: {exc!r}")
            response = build_response("96", "SYSTEM_ERROR")

        conn.sendall(response.encode("utf-8"))
        print(f"Sent response: {response}")

    except Exception as exc:
        print(f"Connection handling error from {addr}: {exc!r}")
    finally:
        try:
            conn.shutdown(socket.SHUT_RDWR)
        except OSError:
            pass
        conn.close()
        print("-" * 50)

def run_server() -> None:
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_sock:

```

```

server_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server_sock.bind((HOST, PORT))
server_sock.listen(5)

print(f"Server listening on {HOST}:{PORT}")

while True:
    conn, addr = server_sock.accept()
    handle_client(conn, addr)

if __name__ == "__main__":
    run_server()

```

Note. Scripts need to be run in different terminal windows on same PC – although with some minor modification you can work with a peer...

Hint: Their IP address!

Server Output	Client Output
Server listening on 127.0.0.1:5000 Connected by ('127.0.0.1', 59478) Raw request: MTI=0100 PAN=4111111111111111 AMOUNT=10075 CUR=GBP Received transaction: PAN=*****1111 AMOUNT=10075 CUR=GBP Sent response: MTI=0110 RC=05 MSG=DECLINED ----- -----	Sending request: MTI=0100 PAN=4111111111111111 AMOUNT=10075 CUR=GBP Received response: MTI=0110 RC=05 MSG=DECLINED



Don't forget to test the client and server with different transaction amounts, good and bad fields etc.

Intermediate Python: Exercise #10

Parallelism

Complexity level	Solo, Pair or Team?	Target time
		 30 minutes

Brief

This exercise presents a problem for you to solve using Python. It is intended to reinforce your understanding of parallel programming concepts and support your preparation for the wider challenges of the programme.

Throughout the process, you are encouraged to share and discuss ideas with your peers, particularly your understanding of the given problem.

After completing the task, don't forget to seek constructive feedback from your instructor.

Comparing your solution with those of your peers is also strongly recommended as different techniques and approaches may come to light which may prove useful to you.

Tools overview

You will be using a variety of software tools:

- **Python 3** (you are recommended to use Python version 3.12)
 - a high-level, general purpose programming language favoured by Data Scientists for experiments
 - <https://www.python.org/>
- **A suitable IDE**, e.g.
 - Microsoft Visual Studio Code
 - PyCharm Community Edition
 - IDLE – but not really advised here due to potential threading concerns
- **A Linux VM**

Solutions should be tested on a Microsoft Windows platform but should work, largely unaltered on others, e.g. Linux distributions, Apple Mac OS X etc., including through the use of containerisation and virtualisation.

Expected outcome(s)

On success, you should be able to confidently solve a parallelised-problem by writing appropriately structured and idiomatic Python 3 code which reflects good practice and follows PEP8 guidelines.

Set Problem

If you recall, previously in Exercise #02, you were asked to create a Python script which created a sequence of unique **transaction reference numbers**.

Typically, they included:

- A timestamp (for ordering & traceability)
- A sequence number (for uniqueness within the same second)
- An Optional checksum (for integrity)

You use the following dash separated format:

PREFIX-YYYYMMDD-HHMMSS-SEQ

Where:

PREFIX	TXN (abbreviation of “transaction”; a default value)
YYYY	Year (4 digits, e.g. 2026)
MM	Month (2 digits, e.g. 07 for July)
DD	Day (2 digits, e.g. 12 for the 12 th day of the month)
HH	Hours (2 digits, 24-hour clock, e.g. 00→12)
MM	Minutes (2 digits, e.g. 00→59)
SS	Seconds (2 digits, e.g. 00→59)
SEQ	A leading zero padded counter of 4 digits , starting at 0001 each second.

Initially you wrote a single function (“transaction_reference_generator”) which **yielded** successive Transaction references, i.e. a function providing a single value, rather than a complete list.

In Exercise #05 you then revised this solution, changing it from a generator to a **custom iterator** class.

Now, you must generate 40 of these transaction reference numbers **in parallel**, using multiple processes to reduce the overall creation time. However, you must also ensure that all transaction numbers are truly unique and do not become duplicated across multiple processes, i.e. avoid “collisions”...

Hints

- *If necessary, please review the tutorial content demonstrated in DG09 and DG10 to help.*
- As this task is CPU-bound, you will need to import the **multiprocessing** module.
- A suggested technique to ensure all transactions are unique is to tag the actual Process ID (PID) into the transaction reference number. *Although that too may have issues...*

Stretch objectives (Things to try)

- Provide the uniqueness of your transaction reference numbers by printing:
 - Number of TRNs generated
 - Number of unique TRNs generated
 - Number of collisions (- hopefully zero!)
 - Ensure that your code is PEP8 compliant by using your IDE or available tooling.
 - Include suitable error handling that prevents the script from crashing if run-time issues occur etc. Correctly report any errors to `stderr`, rather than `stdout`.
 - Use an appropriate linting tool to discover any potential problems with your code, e.g. unnecessary imports, unused variables etc.
 - Consider the use of the `ProcessPoolExecutor` available in `concurrent.futures` and use approximately 4 workers.
-

Additional notes and reflection points

- Which data structures did you select as part of your solution?
 - Did you find a better solution when comparing your solution with your peers?
 - Which part of the solution did you find the most complex to solve?
 - Did you choose EAFP or LBYL error handling techniques?
 - Did you find parallelism a useful feature of Python? Had you used it before?
 - What would be the advantages of using a Process Pool?
-

Intermediate Python: Solution #10

Parallelism

The following is a sample script that covers some of the basic steps of the above practical. There are many possible ways to write code like this, so yours may not look exactly the same but it should be PEP8-compliant and readable.

```
from __future__ import annotations

import os
import time
from concurrent.futures import ProcessPoolExecutor
from datetime import datetime, timezone
from typing import Iterator

def transaction_reference_generator(
    prefix: str,
    node_id: str,
    worker_id: str,
) -> Iterator[str]:
    """
    Multiprocess-safe transaction reference generator.

    Format:
        PREFIX-NODEID-WORKERID-YYYYMMDDTHHMMSSmmm-PID-SEQ

    Example:
        TXN-BANK01-W01-20260405T153210123-04217-0001
    """
    pid = os.getpid()
    last_timestamp_ms = -1
    sequence = 0

    while True:
        timestamp_ms = time.time_ns() // 1_000_000

        if timestamp_ms == last_timestamp_ms:
            sequence += 1
        else:
            last_timestamp_ms = timestamp_ms
            sequence = 1

        dt = datetime.fromtimestamp(timestamp_ms / 1000, tz=timezone.utc)
        ts = dt.strftime("%Y%m%dT%H%M%S") + f"{timestamp_ms % 1000:03d}"

        yield f"{prefix}-{node_id}-{worker_id}-{ts}-{pid:05d}-{sequence:04d}"

def generate_ids(worker_id: str, count: int) -> list[tuple[str, str]]:
    """
    Worker task executed in a process from the pool.
    """
    pid = os.getpid()
    print(f"{worker_id} running in PID {pid}")

    gen = transaction_reference_generator(
        prefix="TXN",
        node_id="BANK01",

```

```

        worker_id=worker_id,
    )

    return [(worker_id, next(gen)) for _ in range(count)]

def demo_process_pool() -> None:
    worker_count = 4
    ids_per_worker = 10

    results: list[tuple[str, str]] = []

    with ProcessPoolExecutor(max_workers=worker_count) as executor:
        futures = [
            executor.submit(generate_ids, f"W{i+1:02d}", ids_per_worker)
            for i in range(worker_count)
        ]

        for future in futures:
            results.extend(future.result())

    ids = [txn_id for _, txn_id in results]
    unique_ids = set(ids)

    print("\nGenerated IDs:")
    for worker_id, txn_id in sorted(results):
        print(f"{worker_id}: {txn_id}")

    print("\nSummary")
    print(f"Total generated: {len(ids)}")
    print(f"Unique generated: {len(unique_ids)}")
    print(f"Collisions found: {len(ids) - len(unique_ids)}")

    if len(ids) == len(unique_ids):
        print("No collisions detected across workers.")

if __name__ == "__main__":
    demo_process_pool()

```

Sample output

```

W01 running in PID 13676
W02 running in PID 13676
W03 running in PID 13676
W04 running in PID 13676

Generated IDs:
W01: TXN-BANK01-W01-20260410T164117940-13676-0001
W01: TXN-BANK01-W01-20260410T164117940-13676-0002
W01: TXN-BANK01-W01-20260410T164117940-13676-0003
W01: TXN-BANK01-W01-20260410T164117940-13676-0004
W01: TXN-BANK01-W01-20260410T164117940-13676-0005
W01: TXN-BANK01-W01-20260410T164117940-13676-0006
W01: TXN-BANK01-W01-20260410T164117940-13676-0007
W01: TXN-BANK01-W01-20260410T164117940-13676-0008
W01: TXN-BANK01-W01-20260410T164117940-13676-0009
W01: TXN-BANK01-W01-20260410T164117940-13676-0010
W02: TXN-BANK01-W02-20260410T164117941-13676-0001
W02: TXN-BANK01-W02-20260410T164117941-13676-0002
W02: TXN-BANK01-W02-20260410T164117941-13676-0003
W02: TXN-BANK01-W02-20260410T164117941-13676-0004
W02: TXN-BANK01-W02-20260410T164117941-13676-0005

```

```
W02: TXN-BANK01-W02-20260410T164117941-13676-0006
W02: TXN-BANK01-W02-20260410T164117941-13676-0007
W02: TXN-BANK01-W02-20260410T164117941-13676-0008
W02: TXN-BANK01-W02-20260410T164117941-13676-0009
W02: TXN-BANK01-W02-20260410T164117941-13676-0010
W03: TXN-BANK01-W03-20260410T164117941-13676-0001
W03: TXN-BANK01-W03-20260410T164117941-13676-0002
W03: TXN-BANK01-W03-20260410T164117941-13676-0003
W03: TXN-BANK01-W03-20260410T164117941-13676-0004
W03: TXN-BANK01-W03-20260410T164117941-13676-0005
W03: TXN-BANK01-W03-20260410T164117941-13676-0006
W03: TXN-BANK01-W03-20260410T164117941-13676-0007
W03: TXN-BANK01-W03-20260410T164117941-13676-0008
W03: TXN-BANK01-W03-20260410T164117941-13676-0009
W03: TXN-BANK01-W03-20260410T164117942-13676-0001
W04: TXN-BANK01-W04-20260410T164117942-13676-0001
W04: TXN-BANK01-W04-20260410T164117942-13676-0002
W04: TXN-BANK01-W04-20260410T164117942-13676-0003
W04: TXN-BANK01-W04-20260410T164117942-13676-0004
W04: TXN-BANK01-W04-20260410T164117942-13676-0005
W04: TXN-BANK01-W04-20260410T164117942-13676-0006
W04: TXN-BANK01-W04-20260410T164117942-13676-0007
W04: TXN-BANK01-W04-20260410T164117942-13676-0008
W04: TXN-BANK01-W04-20260410T164117942-13676-0009
W04: TXN-BANK01-W04-20260410T164117942-13676-0010
```

Summary

Total generated: 40

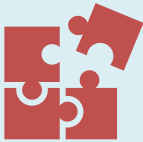
Unique generated: 40

Collisions found: 0

No collisions detected across workers.

Intermediate Python: Exercise #11

Testing

Complexity level	Solo, Pair or Team?	Target time
	 or 	 30 minutes

Brief

This exercise gives you a problem to solve using Python. It is designed to strengthen your core Python skills, while also developing your ability to write effective unit tests and evaluate your script's performance through profiling.

Throughout the process, you are encouraged to share and discuss ideas with your peers, particularly your understanding of the given problem.

After completing the task, don't forget to seek constructive feedback from your instructor.

Comparing your solution with those of your peers is also strongly recommended as different techniques and approaches may come to light which may prove useful to you.

Tools overview

You will be using a variety of software tools:

- **Python 3** (you are recommended to use Python version 3.12)
 - a high-level, general purpose programming language favoured by Data Scientists for experiments
 - <https://www.python.org/>
- **A suitable IDE**, e.g.
 - Microsoft Visual Studio Code
 - PyCharm Community Edition
 - IDLE
- **Unittest and/or pytest frameworks**
 - <https://docs.python.org/3/library/unittest.html#module-unittest>
 - <https://docs.pytest.org/en/stable/>
 - pytest may need to be installed via pip utility (or uv)

Solutions should be tested on a Microsoft Windows platform but should work, largely unaltered on others, e.g. Linux distributions, Apple Mac OS X etc., including through the use of containerisation and virtualisation.

Expected outcome(s)

On success, you should be able to confidently create appropriate unit tests for the functions provided in this lab, first determining their correctness, and then analysing them to see which of two solutions is more efficient (time-wise, at least) and therefore more appropriate for promoting to production code.

Set Problem

Following (what can be best described as) “mixed” feedback during a code review, an experienced Python developer has produced two alternative implementations of a function intended to validate customer account names. This includes common honorifics titles, double-barrelled last names etc.

Code version 1: Regular Expression solution

```
import re

VALID_HONORIFICS = {"mr", "mrs", "miss", "ms", "dr", "prof"}

HONORIFIC_PART = r"(mr|mrs|miss|ms|dr|prof)"
NAME_PART = r"[A-Za-z]{2,}"
LAST_NAME_PART = r"[A-Za-z]{2,}(:[ -][A-Za-z]{2,})?"
ACCOUNT_NAME_RE = re.compile(
    rf"^\s*{HONORIFIC_PART}\s+({NAME_PART})\s+({LAST_NAME_PART})\s*$",
    re.IGNORECASE,
)

def is_valid_account_name_v1(value: str) -> bool:
    if not isinstance(value, str):
        return False
    return ACCOUNT_NAME_RE.fullmatch(value) is not None
```

Code version 2: Mostly string-based solution

```
import re

def is_valid_account_name_v2(value: str) -> bool:
    if not isinstance(value, str):
        return False

    value = value.strip()
    if value == "":
        return False

    pieces = value.split()
    if len(pieces) < 3:
```

```
        return False

    honorific = pieces[0].strip().lower()
    if honorific not in VALID_HONORIFICS:
        return False

    first_name = pieces[1].strip()
    if len(first_name) < 2:
        return False

    if not re.fullmatch(r"[A-Za-z]+", first_name):
        return False

    last_name = " ".join(pieces[2:]).strip()
    if len(last_name) < 2:
        return False

    for ch in last_name:
        if not (ch.isalpha() or ch == " " or ch == "-"):
            return False

    if "--" in last_name or " " in last_name or "- " in last_name or " -" in
last_name:
        return False
    if last_name.startswith("-") or last_name.endswith("-"):
        return False

    if "-" in last_name and " " in last_name:
        return False

    if "-" in last_name:
        subparts = last_name.split("-")
    elif " " in last_name:
        subparts = last_name.split(" ")
    else:
        subparts = [last_name]

    if len(subparts) == 0 or len(subparts) > 2:
        return False

    for part in subparts:
        part = part.strip()
        if len(part) < 2:
            return False
        if not re.fullmatch(r"[A-Za-z]+", part):
            return False

    return True
```

Firstly, you will need to **create** a suitable **unit test** to check the functionality of both functions to ensure that they both work appropriately.

Secondly, you will need to **select** and **apply** a suitable **profiling technique** to check which version is more efficient.

Note. No changes of the original coded solutions are to be made.

Hints

- *You should use either the standard unittest or installed pytest module for completing functional tests; what and how you test is your choice.*
- The choice of profiling technique to use is yours (- but timing seems like a good idea!)
- Brevity of code may or may not result in a more efficient solution...it's not always that obvious from looking at the number of physical/logical SLoC (Source Lines of Code).

Physical = every line

Logical = only actual statements (ignores comments, blank lines etc.)

Stretch objectives (Things to try)

- Swap your chosen unit test framework and try a different one, e.g. use unittest instead of pytest or vice versa.
- Ensure your code is PEP8 compliant by using available tools.
- Try to use a linter to discover potential problems with your code, e.g. unnecessary imports, unused variables etc.
- Can you calculate the code *coverage* of your unit test?

Additional notes and reflection points

- What code aspects have impacted the efficiency of each solution?
 - How did you identify this?
 - Is the answer as straightforward as it initially seems? (- and are you *missing* anything?)
-

Intermediate Python: Solution #11

Testing

The following is a sample script that covers some of the basic steps of the above practical. There are many possible ways to write code like this, so yours may not look exactly the same but it should be PEP8-compliant and readable.

```
import re
import timeit
import unittest
from typing import Iterable

VALID_HONORIFICS = {"mr", "mrs", "miss", "ms", "dr", "prof"}

HONORIFIC_PART = r"(mr|mrs|miss|ms|dr|prof)"
NAME_PART = r"[A-Za-z]{2,}"
LAST_NAME_PART = r"[A-Za-z]{2,}(?:[ -][A-Za-z]{2,})?"
ACCOUNT_NAME_RE = re.compile(
    rf"^\s*{HONORIFIC_PART}\s+({NAME_PART})\s+({LAST_NAME_PART})\s*$",
    re.IGNORECASE,
)

def is_valid_account_name_v1(value: str) -> bool:
    if not isinstance(value, str):
        return False
    return ACCOUNT_NAME_RE.fullmatch(value) is not None

def is_valid_account_name_v2(value: str) -> bool:
    if not isinstance(value, str):
        return False

    value = value.strip()
    if value == "":
        return False

    pieces = value.split()
    if len(pieces) < 3:
        return False

    honorific = pieces[0].strip().lower()
    if honorific not in VALID_HONORIFICS:
        return False

    first_name = pieces[1].strip()
    if len(first_name) < 2:
        return False

    if not re.fullmatch(r"[A-Za-z]+", first_name):
        return False

    last_name = " ".join(pieces[2:]).strip()
    if len(last_name) < 2:
        return False

    for ch in last_name:
        if not (ch.isalpha() or ch == " " or ch == "-"):
            return False
```

```

    if "--" in last_name or " " in last_name or "- " in last_name or " -" in
last_name:
    return False
    if last_name.startswith("-") or last_name.endswith("-"):
    return False

    if "-" in last_name and " " in last_name:
    return False

    if "-" in last_name:
        subparts = last_name.split("-")
    elif " " in last_name:
        subparts = last_name.split(" ")
    else:
        subparts = [last_name]

    if len(subparts) == 0 or len(subparts) > 2:
    return False

    for part in subparts:
        part = part.strip()
        if len(part) < 2:
            return False
        if not re.fullmatch(r"[A-Za-z]+", part):
            return False

    return True

class TestAccountNameValidation(unittest.TestCase):
    def assert_both_true(self, value: str) -> None:
        self.assertTrue(is_valid_account_name_v1(value), f"fast failed for: {value}")
        self.assertTrue(is_valid_account_name_v2(value), f"slow failed for: {value}")

    def assert_both_false(self, value: str) -> None:
        self.assertFalse(is_valid_account_name_v1(value), f"fast passed for:
{value}")
        self.assertFalse(is_valid_account_name_v2(value), f"slow passed for:
{value}")

    def test_valid_simple_names(self) -> None:
        for name in [
            "Mr John Smith",
            "Mrs Jane Brown",
            "Miss Emily Jones",
            "Dr Sarah Connor",
            "Prof Alan Turing",
            "Ms Rachel Green",
        ]:
            with self.subTest(name=name):
                self.assert_both_true(name)

    def test_valid_hyphenated_last_names(self) -> None:
        for name in [
            "Mr John Smith-Jones",
            "Mrs Sarah Brown-Wilson",
            "Dr Peter Taylor-Reed",
        ]:
            with self.subTest(name=name):
                self.assert_both_true(name)

    def test_valid_spaced_last_names(self) -> None:

```

```
for name in [
    "Mr John Van Dyke",
    "Mrs Anna De Vries",
    "Dr Peter Le Roth",
]:
    with self.subTest(name=name):
        self.assert_both_true(name)

def test_invalid_honorific(self) -> None:
    for name in [
        "Sir John Smith",
        "Lord Peter Jones",
        "Mx Alex Brown",
    ]:
        with self.subTest(name=name):
            self.assert_both_false(name)

def test_invalid_first_name_too_short(self) -> None:
    for name in [
        "Mr J Smith",
        "Dr A Brown",
    ]:
        with self.subTest(name=name):
            self.assert_both_false(name)

def test_invalid_last_name_too_short(self) -> None:
    for name in [
        "Mr John S",
        "Mrs Anna B",
        "Dr Peter A-B",
    ]:
        with self.subTest(name=name):
            self.assert_both_false(name)

def test_invalid_characters(self) -> None:
    for name in [
        "Mr John Sm1th",
        "Mrs Jane Br@wn",
        "Dr Sarah O'Neil",
        "Miss Emma Smith_Lee",
    ]:
        with self.subTest(name=name):
            self.assert_both_false(name)

def test_invalid_structure(self) -> None:
    for name in [
        "",
        "Mr",
        "Mr John",
        "John Smith",
        "Mr John Smith Jones Brown",
    ]:
        with self.subTest(name=name):
            self.assert_both_false(name)

def test_invalid_last_name_separator_usage(self) -> None:
    for name in [
        "Mr John Smith--Jones",
        "Mr John Smith - Jones",
        "Mr John -Smith",
        "Mr John Smith-",
        "Mr John Smith-J",
    ]:
```

```

]:
    with self.subTest(name=name):
        self.assert_both_false(name)

    def test_non_string_input(self) -> None:
        for value in [None, 123, 12.5, ["Mr", "John", "Smith"]]:
            with self.subTest(value=value):
                self.assertFalse(is_valid_account_name_v1(value)) # type:
ignore[arg-type]
                self.assertFalse(is_valid_account_name_v2(value)) # type:
ignore[arg-type]

def build_test_dataset() -> list[str]:
    base_values = [
        "Mr John Smith",
        "Mrs Jane Brown",
        "Miss Emily Jones",
        "Dr Sarah Connor",
        "Prof Alan Turing",
        "Ms Rachel Green",
        "Mr John Smith-Jones",
        "Mr John Van Dyke",
        "Sir John Smith",
        "Mr J Smith",
        "Mr John S",
        "Mr John Smith",
        "John Smith",
        "Mr John Smith--Jones",
        "Mr John Smith Jones Brown",
    ]
    return base_values * 2000

def profile_validator(func, dataset: Iterable[str], loops: int = 100) -> float:
    data = list(dataset)
    return timeit.timeit(lambda: [func(item) for item in data], number=loops)

def run_tests() -> bool:
    suite =
unittest.defaultTestLoader.loadTestsFromTestCase(TestAccountNameValidation)
    result = unittest.TextTestRunner(verbosity=2).run(suite)
    return result.wasSuccessful()

def run_profiling() -> None:
    dataset = build_test_dataset()

    fast_time = profile_validator(is_valid_account_name_v1, dataset)
    slow_time = profile_validator(is_valid_account_name_v2, dataset)

    print("\nProfiling results")
    print("-----")
    print(f"Fast version: {fast_time:.6f} seconds")
    print(f"Slow version: {slow_time:.6f} seconds")

    if fast_time < slow_time:
        print(f"Faster function: is_valid_account_name_v1 ({slow_time /
fast_time:.2f}x faster)")
    elif slow_time < fast_time:
        print(f"Faster function: is_valid_account_name_v2 ({fast_time /
slow_time:.2f}x faster)")
    else:
        print("Both functions took the same time")

```

```
if __name__ == "__main__":  
    ok = run_tests()  
    if ok:  
        run_profiling()
```

Spoiler warning!

```
test_invalid_characters (__main__.TestAccountNameValidation.test_invalid_characters) ... ok
test_invalid_first_name_too_short
(__main__.TestAccountNameValidation.test_invalid_first_name_too_short) ... ok
test_invalid_honorific (__main__.TestAccountNameValidation.test_invalid_honorific) ... ok
test_invalid_last_name_separator_usage
(__main__.TestAccountNameValidation.test_invalid_last_name_separator_usage) ... ok
test_invalid_last_name_too_short
(__main__.TestAccountNameValidation.test_invalid_last_name_too_short) ... ok
test_invalid_structure (__main__.TestAccountNameValidation.test_invalid_structure) ... ok
test_non_string_input (__main__.TestAccountNameValidation.test_non_string_input) ... ok
test_valid_hyphenated_last_names
(__main__.TestAccountNameValidation.test_valid_hyphenated_last_names) ... ok
test_valid_simple_names (__main__.TestAccountNameValidation.test_valid_simple_names) ... ok
test_valid_spaced_last_names (__main__.TestAccountNameValidation.test_valid_spaced_last_names)
... ok
```

```
-----
Ran 10 tests in 0.003s
```

OK

Profiling results

```
-----
Fast version: 2.235929 seconds
Slow version: 6.278979 seconds
Faster function: is_valid_account_name_v1 (2.81x faster)
```

Intermediate Python: Exercise #12

FastAPI

Complexity level	Solo, Pair or Team?	Target time
	 or 	 40 minutes

Brief

This exercise presents a problem for you to solve using a web service framework; we've selected FastAPI but many others are available, e.g. Flask, Django, etc. It is intended to reinforce your understanding of designing and implementing a simple API using defined endpoints, HTTP methods and suitable handlers to process requests (safely), query backend resources and then package appropriate responses.

Throughout the process, you are encouraged to share and discuss ideas with your peers, particularly your understanding of the given problem.

After completing the task, don't forget to seek constructive feedback from your instructor.

Comparing your solution with those of your peers is also strongly recommended as different techniques and approaches may come to light which may prove useful to you.

Tools overview

You will be using a variety of software tools:

- **Python 3** (you are recommended to use Python version 3.12)
 - a high-level, general purpose programming language favoured by Data Scientists for experiments
 - <https://www.python.org/>
- **FastAPI**
 - High-performance Python web framework for building APIs, based on standard Python type hints
 - Automatic data validation & parsing using Python typing and Pydantic
 - Auto-generated interactive docs (Swagger UI & OpenAPI) out of the box
 - Easy async support for handling concurrent requests efficiently
 - Simple, clean syntax that makes building and maintaining APIs fast and readable
 - <https://fastapi.tiangolo.com/>
- **Pandas (optional)**
 - Pandas is the standard Python DataFrame library for data analysis and manipulation
 - Best for EDA, notebooks, and small-to-medium in-memory datasets
 - Mature ecosystem with extensive features (groupby, joins, time series)
 - Single-threaded and slower on large datasets; higher memory usage

- Eager execution model with no query optimisation
- <https://pandas.pydata.org/>
- **Polars (optional)**
 - Polars is a high-performance DataFrame library written in Rust with Python bindings
 - Best for large datasets, ETL pipelines, and backend/data engineering workloads
 - Fast and memory-efficient with built-in parallelism
 - Supports lazy execution with query optimisation
 - Smaller ecosystem and slight learning curve vs Pandas
 - <https://pola.rs/>
- **A suitable IDE, e.g.**
 - Microsoft Visual Studio
 - PyCharm Community Edition
 - IDLE

Solutions should be tested on a Microsoft Windows platform but should work, largely unaltered on others, e.g. Linux distributions, Apple Mac OS X etc., including through the use of containerisation and virtualisation.

Expected outcome(s)

On success, you should be able to confidently read through the outline of a basic API service and implement it using FastAPI.

Set Problem

You are asked to build a small FastAPI service that initially loads a synthetic financial transactions CSV into a Pandas (or Polars) DataFrame and exposes query endpoints for transaction lookup as a web-based API.

To help you get started, we've provided a synthetic CSV file of transactions:

- Filename: `bank_transactions_synthetic.csv`
- Rows: 100
- CSV columns:
 - `transaction_id` — unique transaction identifier
 - `account_number` — bank account number as text; duplicates are expected
 - `transaction_date` — ISO date in YYYY-MM-DD
 - `transaction_time` — time in HH:MM:SS
 - `transaction_type` — either deposit or withdrawal
 - `amount_gbp` — transaction amount in GBP between 4.50 and 500.00



The CSV *deliberately* includes:

- duplicate account numbers
- a few missing or invalid account numbers such as blank values or NaN

Data Sample (head):

```
transaction_id,account_number,transaction_date,transaction_time,transaction_type,amount_gbp
TXN0001,66306997,2026-01-21,15:41:36,withdrawal,142.19
TXN0002,28728463,2026-02-13,09:51:38,deposit,192.75
TXN0003,66629388,2026-02-14,18:59:25,withdrawal,404.43
TXN0004,83140807,2026-02-28,17:45:42,deposit,486.68
TXN0005,14265799,2026-01-11,18:02:58,withdrawal,415.47
TXN0006,13561597,2026-02-16,18:30:37,deposit,353.62
TXN0007,NaN,2026-03-26,12:08:55,withdrawal,492.68
```

Desired Behaviours

You may choose your own route names, but the API must contain exactly these three endpoint types:

1. Health endpoint

A simple endpoint to confirm the service is actually “up”, listening, and responding to a request.

Example:

- **GET /health**

Expected JSON shape for response:

```
{
  "status_code": 200,
  "status": "ok"
}
```

2. Transactions by account number

Returns all transactions for a given account number.

Example:

- **GET /transactions?account_number=12345678**

Requirements:

- Account number is required
- You should clean and validate the incoming account number before querying

- Do not match blank/NaN account numbers
- Return all matching records

Expected JSON shape for response:

```
{
  "status_code": 200,
  "account_number": "12345678",
  "count": 3,
  "transactions": [
    {
      "transaction_id": "TXN0001",
      "account_number": "12345678",
      "transaction_date": "2026-01-08",
      "transaction_time": "09:15:44",
      "transaction_type": "deposit",
      "amount_gbp": 125.40
    }
  ]
}
```

3. Transactions by account number and transaction flag

Returns transactions for a supplied account number filtered by transaction type.

Example:

- GET /transactions/filter?account_number=12345678&transaction_type=deposit

Requirements:

- Account number is required
- Transaction type is required
- Transaction type must *only* accept either:
 - deposit
 - withdrawal
- Clean both inputs before querying
- Return filtered matching records as JSON

Expected JSON shape for response:

```
{
  "status_code": 200,
  "account_number": "12345678",
  "transaction_type": "deposit",
  "count": 2,
  "transactions": []
}
```

```
}
```

**Suggested cleaning rules:**

Before querying with Pandas (or Polars), clean the dataset and incoming query values.

At the very minimum we would suggest:

- ✓ Trim surrounding whitespace
- ✓ Treat any blank strings and `NaN` account numbers as null/missing
- ✓ Ensure account numbers are handled as strings, not integers
- ✓ Normalise transaction type to lowercase
- ✓ Reject invalid query values with a 4xx response

Hints

- Keep the CSV loading logic separate from the endpoint handlers.
- Use Pandas/Polars filtering techniques rather than Python loops.
- You may use eager Polars DataFrames.
- It is acceptable to load the CSV once at startup and reuse it for all requests.
- Response bodies must be JSON serialisable.

Stretch objectives (Things to try)

19. Ensure that your code is PEP8 compliant by using your IDE or available tooling.
20. Include suitable error that prevents the script from crashing if run-time issues occur etc. Correctly report any errors to `stderr`, rather than `stdout`.
21. Use an appropriate linting tool to discover any potential problems with your code, e.g. unnecessary imports, unused variables etc.
22. Did you find any logic gaps?

Additional notes and reflection points

- Which data structures did you select as part of your solution?
- Did you find a better solution when comparing your solution with your peers?
- Which part of the solution did you find the most complex to solve?
- Did you choose EAFP or LBYL error handling techniques?

Advanced Python: Solution #12

FastAPI

The following is a sample script that covers some of the basic steps of the above practical. There are many possible ways to write code like this, so yours may not look exactly the same but it should be PEP8-compliant and readable.

```
from __future__ import annotations

from pathlib import Path
from typing import Any

import polars as pl
from fastapi import FastAPI, Query
from fastapi.responses import JSONResponse

BASE_DIR = Path(__file__).resolve().parent
DATA_FILE = BASE_DIR / "bank_transactions_synthetic.csv"
VALID_TRANSACTION_TYPES = {"deposit", "withdrawal"}
MISSING_ACCOUNT_VALUES = {"", "nan", "none", "null"}

app = FastAPI(title="Transaction Query Service", version="1.0.0")

def build_json_response(
    status_code: int,
    payload: dict[str, Any],
) -> JSONResponse:
    """Returns a JSON response with a mirrored status code in the body."""
    body = {"status_code": status_code, **payload}
    return JSONResponse(status_code=status_code, content=body)

def clean_account_number(value: Any) -> str | None:
    """Normalises a raw account number value to an 8-digit string.

    Blank-like values such as empty strings, ``NaN``, ``null``, and ``None``
    are treated as missing.
    """
    if value is None:
        return None

    cleaned = str(value).strip()
    if not cleaned:
        return None

    if cleaned.lower() in MISSING_ACCOUNT_VALUES:
        return None
```

```

    if cleaned.endswith(".0"):
        cleaned = cleaned[:-2]

    if not cleaned.isdigit():
        return None

    if len(cleaned) != 8:
        return None

    return cleaned

def clean_transaction_type(value: Any) -> str | None:
    """Normalises and validates the transaction type."""
    if value is None:
        return None

    cleaned = str(value).strip().lower()
    if cleaned in VALID_TRANSACTION_TYPES:
        return cleaned

    return None

def load_transactions(data_file: Path = DATA_FILE) -> pl.DataFrame:
    """Loads the CSV and cleans it before any filtering takes place."""
    dataframe = pl.read_csv(data_file)

    return dataframe.with_columns(
        [
            pl.col("account_number")
                .map_elements(clean_account_number, return_dtype=pl.String)
                .alias("account_number"),
            pl.col("transaction_type")
                .map_elements(clean_transaction_type, return_dtype=pl.String)
                .alias("transaction_type"),
            pl.col("amount_gbp").cast(pl.Float64).round(2).alias("amount_gbp"),
        ]
    )

TRANSACTIONS_DF = load_transactions()

def query_transactions(
    account_number: str,
    transaction_type: str | None = None,
) -> list[dict[str, Any]]:
    """Queries cleaned transactions using Polars expressions."""
    filter_expression = pl.col("account_number") == account_number

```

```

    if transaction_type is not None:
        filter_expression = filter_expression & (
            pl.col("transaction_type") == transaction_type
        )

    return (
        TRANSACTIONS_DF.filter(
            pl.col("account_number").is_not_null()
            & pl.col("transaction_type").is_not_null()
            & filter_expression
        )
        .sort(["transaction_date", "transaction_time", "transaction_id"])
        .to_dicts()
    )

@app.get("/health")
def health() -> JSONResponse:
    """Returns the health status for the service."""
    return build_json_response(200, {"status": "ok"})

@app.get("/transactions")
def get_transactions(
    account_number: str = Query(
        ...,
        description="8-digit bank account number",
    ),
) -> JSONResponse:
    """Returns all transactions for a cleaned account number."""
    cleaned_account_number = clean_account_number(account_number)
    if cleaned_account_number is None:
        return build_json_response(
            400,
            {"error": "account_number must be an 8-digit numeric string"},
        )

    transactions = query_transactions(account_number=cleaned_account_number)
    if not transactions:
        return build_json_response(
            404,
            {
                "error": "No transactions found for the supplied account_number",
                "account_number": cleaned_account_number,
            },
        )

    return build_json_response(
        200,
        {
            "account_number": cleaned_account_number,

```

```

        "count": len(transactions),
        "transactions": transactions,
    },
)

@app.get("/transactions/filter")
def get_transactions_by_type(
    account_number: str = Query(
        ...,
        description="8-digit bank account number",
    ),
    transaction_type: str = Query(
        ...,
        description="Transaction type: deposit or withdrawal",
    ),
) -> JSONResponse:
    """Returns transactions for an account number filtered by type."""
    cleaned_account_number = clean_account_number(account_number)
    if cleaned_account_number is None:
        return build_json_response(
            400,
            {"error": "account_number must be an 8-digit numeric string"},
        )

    cleaned_transaction_type = clean_transaction_type(transaction_type)
    if cleaned_transaction_type is None:
        return build_json_response(
            400,
            {"error": "transaction_type must be 'deposit' or 'withdrawal'"},
        )

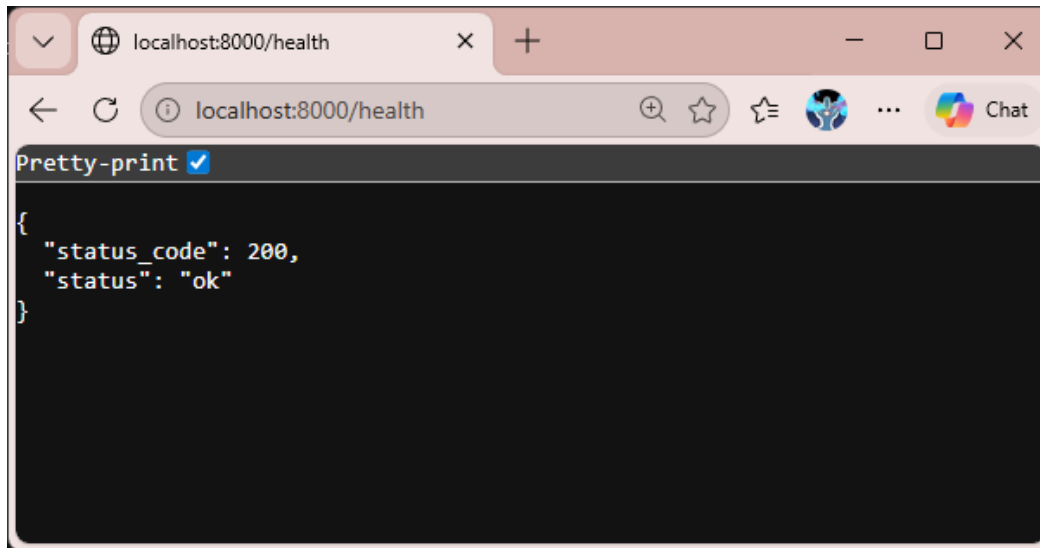
    transactions = query_transactions(
        account_number=cleaned_account_number,
        transaction_type=cleaned_transaction_type,
    )
    if not transactions:
        return build_json_response(
            404,
            {
                "error": (
                    "No transactions found for the supplied account_number "
                    "and transaction_type"
                ),
                "account_number": cleaned_account_number,
                "transaction_type": cleaned_transaction_type,
            },
        )

    return build_json_response(
        200,

```

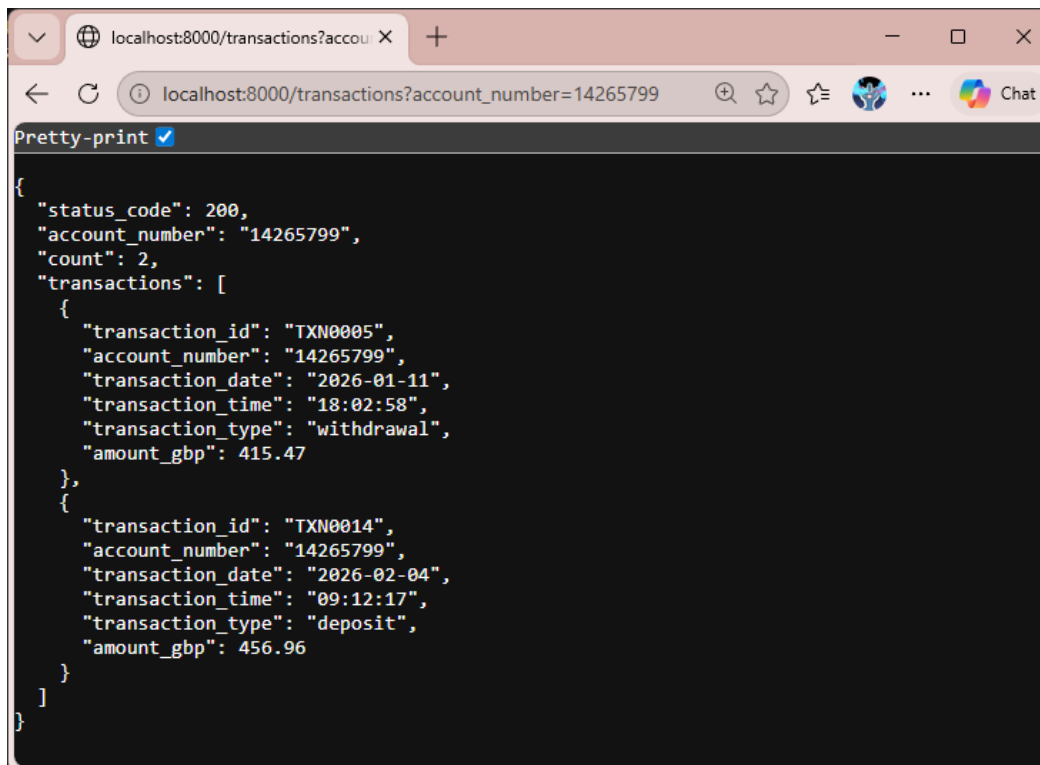
```
{
    "account_number": cleaned_account_number,
    "transaction_type": cleaned_transaction_type,
    "count": len(transactions),
    "transactions": transactions,
},
)
```

Sample Outputs:



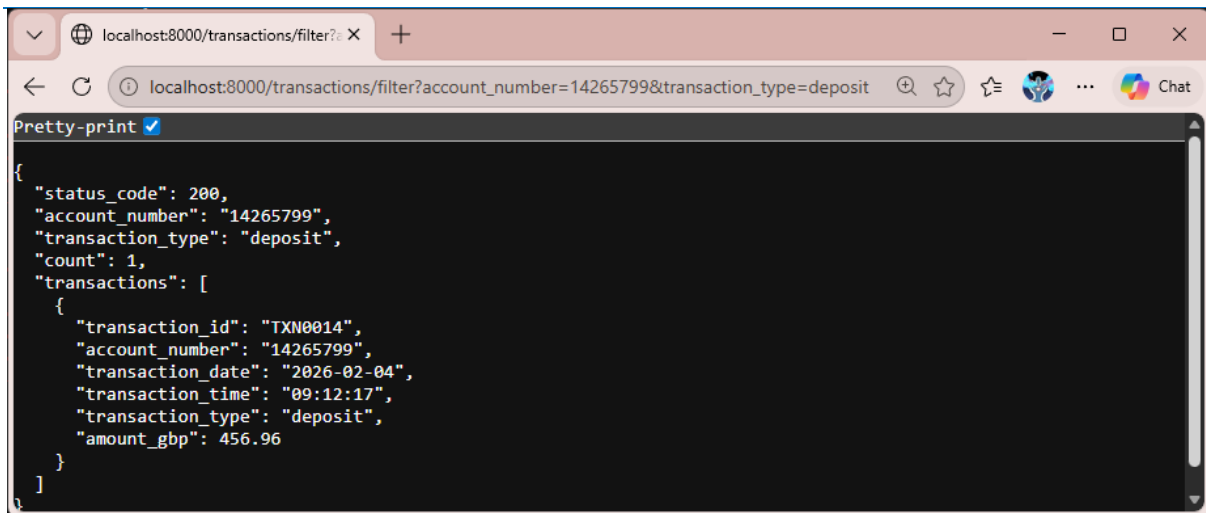
A screenshot of a web browser window with the address bar showing 'localhost:8000/health'. The page content displays a JSON response in a 'Pretty-print' view. The response indicates a successful status with a 200 status code and 'ok' status.

```
{
  "status_code": 200,
  "status": "ok"
}
```



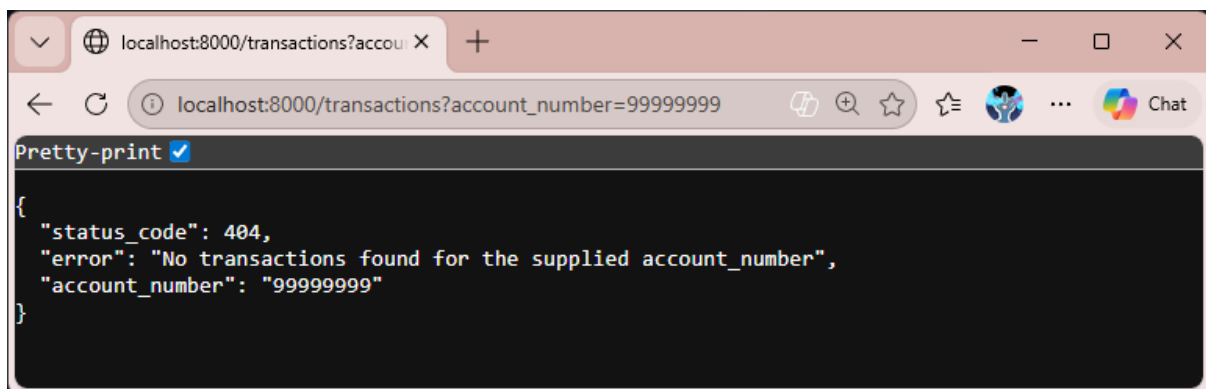
A screenshot of a web browser window with the address bar showing 'localhost:8000/transactions?account_number=14265799'. The page content displays a JSON response in a 'Pretty-print' view. The response includes the account number, a count of 2 transactions, and a list of two transaction objects with details like transaction ID, date, time, type, and amount.

```
{
  "status_code": 200,
  "account_number": "14265799",
  "count": 2,
  "transactions": [
    {
      "transaction_id": "TXN0005",
      "account_number": "14265799",
      "transaction_date": "2026-01-11",
      "transaction_time": "18:02:58",
      "transaction_type": "withdrawal",
      "amount_gbp": 415.47
    },
    {
      "transaction_id": "TXN0014",
      "account_number": "14265799",
      "transaction_date": "2026-02-04",
      "transaction_time": "09:12:17",
      "transaction_type": "deposit",
      "amount_gbp": 456.96
    }
  ]
}
```



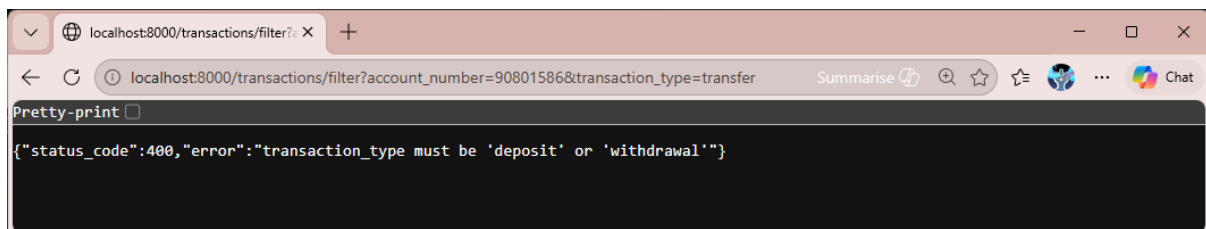
A web browser window showing a GET request to `localhost:8000/transactions/filter?account_number=14265799&transaction_type=deposit`. The response is a JSON object displayed in a 'Pretty-print' view. The JSON contains a status code of 200, the account number, transaction type, count of 1, and a list of transaction details including transaction ID, date, time, and amount.

```
{
  "status_code": 200,
  "account_number": "14265799",
  "transaction_type": "deposit",
  "count": 1,
  "transactions": [
    {
      "transaction_id": "TXN0014",
      "account_number": "14265799",
      "transaction_date": "2026-02-04",
      "transaction_time": "09:12:17",
      "transaction_type": "deposit",
      "amount_gbp": 456.96
    }
  ]
}
```



A web browser window showing a GET request to `localhost:8000/transactions?account_number=99999999`. The response is a JSON object displayed in a 'Pretty-print' view. The JSON contains a status code of 404 and an error message stating that no transactions were found for the supplied account number.

```
{
  "status_code": 404,
  "error": "No transactions found for the supplied account_number",
  "account_number": "99999999"
}
```



A web browser window showing a GET request to `localhost:8000/transactions/filter?account_number=90801586&transaction_type=transfer`. The response is a JSON object displayed in a 'Pretty-print' view. The JSON contains a status code of 400 and an error message stating that the transaction type must be 'deposit' or 'withdrawal'.

```
{
  "status_code": 400,
  "error": "transaction_type must be 'deposit' or 'withdrawal'"
}
```

_Intermediate Python: Exercise #13

Asyncio and testing

Complexity level	Solo, Pair or Team?	Target time
	 or	 70 minutes

Brief

This exercise gives you a problem to solve using Python. It is designed to strengthen your core Python skills, while also developing your ability to write effective code using the `asyncio` library which makes I/O bound code more efficient, without the need to worry about traditional Python multi-threading issues. There is an additional challenge which will require you to create a set of unit tests using `pytest_asyncio` to ensure your code works correctly.

Throughout the process, you are encouraged to share and discuss ideas with your peers, particularly your understanding of the given problem.

After completing the task, don't forget to seek constructive feedback from your instructor.

Comparing your solution with those of your peers is also strongly recommended as different techniques and approaches may come to light which may prove useful to you.

Tools overview

You will be using a variety of software tools:

- **Python 3** (you are recommended to use Python version 3.12)
 - a high-level, general purpose programming language favoured by Data Scientists for experiments
 - <https://www.python.org/>
- **A suitable IDE**, e.g.
 - Microsoft Visual Studio Code
 - PyCharm Community Edition
 - IDLE – but not really advised here due to potential threading concerns
- **asyncio library**
 - A Python library used for writing concurrent code using the **`async/await`** syntax
 - <https://docs.python.org/3/library/asyncio.html>
- **pytest framework**
 - <https://docs.pytest.org/en/stable/>

Solutions should be tested on a Microsoft Windows platform but should work, largely unaltered on others, e.g. Linux distributions, Apple Mac OS X etc., including through the use of containerisation and virtualisation.

Expected outcome(s)

On success, you should be able to use the `asyncio` library to create one (or more) producer coroutines that generate data items, placing them into a single `asyncio.Queue` and have other consumer routines which retrieve data from that queue and process it. In addition you will have used the popular `pytest` framework to test concurrent code written using the `asyncio` library.

Set Problem

You will need to create a Python solution which uses the `asyncio` library to query (in parallel) our previously created FastAPI transaction API, storing its responses into a single `asyncio.Queue` object.

Initially try a sample of 5 different account numbers.

Once all requests have been made and responses stored, the queue can be emptied and the results printed on screen in a “pretty” format, i.e. structured human-readable, not machine-readable raw JSON.

Finally, you will need to create appropriate unit tests using `pytest` to check the core functionality (and correctness) of your solution.

Hints

- Your code will need functions flagged with the `async` keyword and these must be awaited.
- You’ll need to import at least two functions from the previous exercise: `clean_account_number` and `query_transactions`.
- You’ll need to create an `asyncio.Queue` object to store a list of dictionaries (storing string: Any pairs) – this will reflect the JSON responses.
- If you’re not happy with the printing order from the queue, you’ll need to modify the order of the responses...
- You’ll need to import `pprint` module for the `pprint` function.
- You will need to manually place a manually crafted “empty” response in your queue if the account number passed is invalid, i.e. don’t trouble the `query_transactions` function as it’s an unnecessary API request.
- You’ll need to import `asyncio`, `pytest` and key functions and use them to ensure all core functionality is successfully tested.

Stretch objectives (Things to try)

- Ensure your code is PEP8 compliant by using available tools.
 - Try to use a linter to discover potential problems with your code, e.g. unnecessary imports, unused variables etc.
-

Additional notes and reflection points

- Which features of pytest's asyncio support did you use?
 - What code aspects have impacted the efficiency of each solution?
 - How did you identify this?
 - Is the answer as straightforward as it initially seems? (- and are you *missing* anything?)
-

Intermediate Python: Solution #13

Asyncio

The following is a sample script that covers some of the basic steps of the above practical. There are many possible ways to write code like this, so yours may not look exactly the same but it should be PEP8-compliant and readable.

```
from __future__ import annotations

import asyncio
from pprint import pprint
from typing import Any

from EG12_Solution import clean_account_number, query_transactions

async def fetch_transactions_for_account(
    account_number: str,
    result_queue: asyncio.Queue[dict[str, Any]],
) -> None:
    """Query transactions for one account number and put the result in our
    queue."""
    cleaned_account_number = clean_account_number(account_number)

    if cleaned_account_number is None:
        await result_queue.put(
            {
                "account_number": account_number,
                "status_code": 400,
                "error": "account_number must be an 8-digit numeric string",
                "count": 0,
                "transactions": [],
            }
        )
        return

    transactions = await asyncio.to_thread(
        query_transactions,
        cleaned_account_number,
    )

    if not transactions:
        await result_queue.put(
            {
                "account_number": cleaned_account_number,
                "status_code": 404,
                "error": "No transactions found for the supplied account_number",
                "count": 0,
```

```

        "transactions": [],
    }
)
return

await result_queue.put(
    {
        "account_number": cleaned_account_number,
        "status_code": 200,
        "count": len(transactions),
        "transactions": transactions,
    }
)

async def fetch_transactions_for_accounts(
    account_numbers: list[str],
) -> asyncio.Queue[dict[str, Any]]:
    """Run account queries concurrently and return a queue of results."""
    result_queue: asyncio.Queue[dict[str, Any]] = asyncio.Queue()

    tasks = [
        asyncio.create_task(
            fetch_transactions_for_account(account_number, result_queue)
        )
        for account_number in account_numbers
    ]

    await asyncio.gather(*tasks)
    return result_queue

def drain_queue(
    result_queue: asyncio.Queue[dict[str, Any]],
) -> list[dict[str, Any]]:
    """Pull all items from the queue into a list."""
    results: list[dict[str, Any]] = []

    while not result_queue.empty():
        results.append(result_queue.get_nowait())

    return results

def pretty_print_results(results: list[dict[str, Any]]) -> None:
    """Pretty print the collected query results."""
    for index, result in enumerate(results, start=1):
        print(f"\nResult {index}")
        print("-" * 60)
        pprint(result, sort_dicts=False)

```

```
async def main() -> None:
    """Run five account-number queries in parallel and print the results."""
    account_numbers = [
        "10872248",
        "44187304",
        "25001881",
        "99999999",
        "bad-input",
    ]

    result_queue = await fetch_transactions_for_accounts(account_numbers)
    results = drain_queue(result_queue)
    pretty_print_results(results)

if __name__ == "__main__":
    asyncio.run(main())
```

Output:

```
python .\EG12_Solution.py
```

Result 1

```
-----
{'account_number': 'bad-input',
 'status_code': 400,
 'error': 'account_number must be an 8-digit numeric string',
 'count': 0,
 'transactions': []}
```

Result 2

```
-----
{'account_number': '44187304',
 'status_code': 404,
 'error': 'No transactions found for the supplied account_number',
 'count': 0,
 'transactions': []}
```

Result 3

```
-----
{'account_number': '25001881',
 'status_code': 404,
 'error': 'No transactions found for the supplied account_number',
 'count': 0,
 'transactions': []}
```

Result 4

```
-----
{'account_number': '99999999',
 'status_code': 404,
 'error': 'No transactions found for the supplied account_number',
 'count': 0,
 'transactions': []}
```

Result 5

```
-----
```

```
{'account_number': '10872248',
 'status_code': 200,
 'count': 5,
 'transactions': [{ 'transaction_id': 'TXN0070',
                    'account_number': '10872248',
                    'transaction_date': '2026-01-07',
                    'transaction_time': '09:40:48',
                    'transaction_type': 'withdrawal',
                    'amount_gbp': 415.44},
                  { 'transaction_id': 'TXN0079',
                    'account_number': '10872248',
                    'transaction_date': '2026-02-12',
                    'transaction_time': '15:29:42',
                    'transaction_type': 'deposit',
                    'amount_gbp': 136.7},
                  { 'transaction_id': 'TXN0033',
                    'account_number': '10872248',
                    'transaction_date': '2026-03-11',
                    'transaction_time': '11:39:42',
                    'transaction_type': 'withdrawal',
                    'amount_gbp': 202.2},
                  { 'transaction_id': 'TXN0012',
                    'account_number': '10872248',
                    'transaction_date': '2026-03-23',
                    'transaction_time': '18:08:20',
                    'transaction_type': 'deposit',
                    'amount_gbp': 343.73},
                  { 'transaction_id': 'TXN0016',
                    'account_number': '10872248',
                    'transaction_date': '2026-03-24',
                    'transaction_time': '16:21:11',
                    'transaction_type': 'deposit',
                    'amount_gbp': 135.75}]}
```

The following is a sample script that covers the testing elements of the above practical. There are many possible ways to write code like this, so yours may not look exactly the same but it should be PEP8-compliant and readable.

```
from __future__ import annotations
import asyncio
import pytest

from EG13_Solution import (
    drain_queue,
    fetch_transactions_for_account,
    fetch_transactions_for_accounts,
)

@pytest.mark.asyncio
async def test_fetch_transactions_for_account_success() -> None:
    result_queue: asyncio.Queue[dict[str, object]] = asyncio.Queue()

    await fetch_transactions_for_account("10872248", result_queue)

    result = result_queue.get_nowait()
    assert result["status_code"] == 200
    assert result["account_number"] == "10872248"
    assert result["count"] >= 5
    assert isinstance(result["transactions"], list)

@pytest.mark.asyncio
async def test_fetch_transactions_for_account_invalid_input() -> None:
    result_queue: asyncio.Queue[dict[str, object]] = asyncio.Queue()

    await fetch_transactions_for_account("bad-input", result_queue)

    result = result_queue.get_nowait()
    assert result["status_code"] == 400
    assert result["account_number"] == "bad-input"
    assert result["count"] == 0
    assert result["transactions"] == []

@pytest.mark.asyncio
async def test_fetch_transactions_for_account_not_found() -> None:
    result_queue: asyncio.Queue[dict[str, object]] = asyncio.Queue()

    await fetch_transactions_for_account("99999999", result_queue)

    result = result_queue.get_nowait()
    assert result["status_code"] == 404
    assert result["account_number"] == "99999999"
    assert result["count"] == 0
    assert result["transactions"] == []

@pytest.mark.asyncio
async def test_fetch_transactions_for_accounts_returns_all_results() -> None:
    account_numbers = [
        "10872248",
        "44187304",
        "99999999",
        "bad-input",
        "25001881",
    ]

    result_queue = await fetch_transactions_for_accounts(account_numbers)
```

```

results = drain_queue(result_queue)

assert len(results) == 5
returned_accounts = {result["account_number"] for result in results}
assert "10872248" in returned_accounts
assert "44187304" in returned_accounts
assert "25001881" in returned_accounts
assert "99999999" in returned_accounts
assert "bad-input" in returned_accounts

def test_drain_queue_returns_all_items() -> None:
    result_queue: asyncio.Queue[dict[str, object]] = asyncio.Queue()

    result_queue.put_nowait({"account_number": "11111111", "status_code": 200})
    result_queue.put_nowait({"account_number": "22222222", "status_code": 404})

    results = drain_queue(result_queue)

    assert len(results) == 2
    assert result_queue.empty()

```

Sample test run and outputs

```

pytest -v .\EG13_Solution.py
===== test session starts =====
platform win32 -- Python 3.11.6, pytest-8.3.3, pluggy-1.5.0 -- C:\Program
Files\Python311\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\capat\OneDrive\Desktop\qa\LBGIntPythonExOnly
plugins: Faker-22.1.0, asyncio-0.24.0, anyio-4.13.0, time-machine-2.13.0
asyncio: mode=Mode.STRICT, default_loop_scope=None
collected 5 items

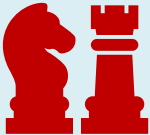
EG13_Solution.py::test_fetch_transactions_for_account_success PASSED [ 20%]
EG13_Solution.py::test_fetch_transactions_for_account_invalid_input PASSED [ 40%]
EG13_Solution.py::test_fetch_transactions_for_account_not_found PASSED [ 60%]
EG13_Solution.py::test_fetch_transactions_for_accounts_returns_all_results PASSED [ 80%]
EG13_Solution.py::test_drain_queue_returns_all_items PASSED [100%]

===== 5 passed in 0.96s =====

```

Intermediate Python: Exercise #14

Extending Python – First Steps

Complexity level	Solo, Pair or Team?	Target time
	 or 	 30 minutes

Brief

This exercise presents a problem for you to solve using a combination of two programming languages: Python and C. It is intended to reinforce your understanding of extending Python using modules written in faster (and typically more efficient) programming languages.

Throughout the process, you are encouraged to share and discuss ideas with your peers, particularly your understanding of the given problem.

After completing the task, don't forget to seek constructive feedback from your instructor.

Comparing your solution with those of your peers is also strongly recommended as different techniques and approaches may come to light which may prove useful to you.

Tools overview

You will be using a variety of software tools:

- **C**
 - A low-level, high performance procedural programming language (C99 is fine)
 - Highly portable and widely used
 - <https://www.learn-c.org/>
- **Python 3** (you are recommended to use Python version 3.12)
 - a high-level, general purpose programming language favoured by Data Scientists for experiments
 - <https://www.python.org/>
- **toml**
 - Tom's Obvious, Minimal Language
 - A popular configuration file format designed to be easy to read/write while mapping unambiguously to data structures; highly reminiscent of an .INI file.
 - Supports nested tables, arrays, and strong typing (ints, floats, strings, booleans, dates).
 - Typically used for project configuration, e.g. Python pyproject.toml, Rust's Cargo.toml, application settings, and cross-platform tooling.
 - <https://toml.io/en/>

- **A suitable IDE**, e.g.
 - Microsoft Visual Studio
 - PyCharm Community Edition
 - IDLE
- **A Linux VM**
 - Your active UID will need access to the sudo command to complete the installation process

Solutions should be tested on a Microsoft Windows platform but should work, largely unaltered on others, e.g. Linux distributions, Apple Mac OS X etc., including through the use of containerisation and virtualisation.

Expected outcome(s)

Once you've succeeded, you'll be able to extend Python by building a module written in C. This mirrors how compiled languages are used to improve performance in widely used libraries such as pandas (C) and polars (Rust), particularly in machine learning and data processing contexts.

Set Problem

In financial systems, it's common to calculate how an investment or loan grows over time when interest is applied repeatedly. This is known as **compound interest**, where interest is calculated not just on the initial amount, but also on previously accumulated interest.

In this task, you will write a function in C that calculates the future value of an investment using the compound interest formula. This type of calculation underpins many real-world systems, from savings accounts to financial analytics tools.

However, this time, rather than coding in Python, we'd like you to implement it as a **C extension for Python**.

Compound interest is calculated using the formula:

$$A = P * (1 + r / n)^{(n * t)}$$

Where:

- **P** = principal (initial amount of money)
- **r** = annual interest rate (as a decimal, e.g. 0.05 for 5%)
- **n** = number of times interest is compounded per year
- **t** = number of years
- **A** = final amount after interest

To help you get started, here's the **C code** required for your extension module:

```
static PyObject *py_compound_interest(PyObject *self, PyObject *args)
{
    double principal;
    double rate;
    int times_per_year;
    double years;

    if (!PyArg_ParseTuple(args, "ddid", &principal, &rate, &times_per_year,
&years)) {
        return NULL;
    }

    if (times_per_year <= 0) {
        PyErr_SetString(PyExc_ValueError, "times_per_year must be greater than
0");
        return NULL;
    }

    if (principal < 0.0) {
        PyErr_SetString(PyExc_ValueError, "principal must be non-negative");
        return NULL;
    }

    if (years < 0.0) {
        PyErr_SetString(PyExc_ValueError, "years must be non-negative");
        return NULL;
    }

    double amount = principal * pow(1.0 + (rate / (double)times_per_year),
                                   (double)times_per_year * years);

    return PyFloat_FromDouble(amount);
}
```

Note. We've added some basic argument validation, from a good practice perspective.

Hints

To successfully create the C module, you will need to implement several key components:

1. A Method Resolution Table (MRT), including a method definition for your function
2. A module definition function
3. A module initialisation function

You will then need to add code for the necessary build and installation steps to compile and install the module.

Once successfully built, try to import and use the function in Python.

If needed, refer back to the step-by-step tutorial in DG14 for guidance.

Stretch objectives (Things to try)

- Ensure your code is PEP8 compliant by using available tools.
 - Try to use a linter to discover potential problems with your code, e.g. unnecessary imports, unused variables etc.
-

Additional notes and reflection points

- What code aspects have impacted the efficiency of each solution?
 - How did you identify this?
 - Is the answer as straightforward as it initially seems? (- and are you *missing* anything?)
-

Intermediate Python: Solution #14

Extending Python – First Steps

The following is the **setup.py** file required to build the python extension module we created in this chapter.

```
from setuptools import Extension, setup

setup(
    name="finance",
    version="1.0.0",
    description="Finance C extension for Python",
    ext_modules=[Extension("finance", ["EG14_Solution_finance_module.c"])],
)
```

The following, “**finance_module.c**” is a sample script that covers some of the basic steps of the above practical. There are many possible ways to write code like this, so yours may not look exactly the same but it should be PEP8-compliant and readable.

```
#define PY_SSIZE_T_CLEAN
#include <Python.h>
#include <math.h>

/*
 * compound_interest(principal, rate, times_per_year, years) -> float
 *
 * Formula:
 *   A = P * (1 + r / n)^(n * t)
 *
 * Where:
 *   P = principal
 *   r = annual interest rate as a decimal, e.g. 0.05 for 5%
 *   n = number of times interest is compounded per year
 *   t = number of years
 */
static PyObject *py_compound_interest(PyObject *self, PyObject *args)
{
    double principal;
    double rate;
    int times_per_year;
    double years;

    if (!PyArg_ParseTuple(args, "ddid", &principal, &rate, &times_per_year,
&years)) {
        return NULL;
    }

    if (times_per_year <= 0) {
```

```

        PyErr_SetString(PyExc_ValueError, "times_per_year must be greater than
0");
        return NULL;
    }

    if (principal < 0.0) {
        PyErr_SetString(PyExc_ValueError, "principal must be non-negative");
        return NULL;
    }

    if (years < 0.0) {
        PyErr_SetString(PyExc_ValueError, "years must be non-negative");
        return NULL;
    }

    double amount = principal * pow(1.0 + (rate / (double)times_per_year),
                                    (double)times_per_year * years);

    return PyFloat_FromDouble(amount);
}

static PyMethodDef FinanceMethods[] = {
    {
        "compound_interest",
        py_compound_interest,
        METH_VARARGS,
        PyDoc_STR(
            "compound_interest(principal, rate, times_per_year, years)\n"
            "--\n\n"
            "Calculate compound interest.\n\n"
            "Arguments:\n"
            "  principal: initial amount\n"
            "  rate: annual interest rate as decimal (e.g. 0.05)\n"
            "  times_per_year: compounding frequency per year\n"
            "  years: number of years\n\n"
            "Returns:\n"
            "  Final amount after compounding."
        )
    },
    {NULL, NULL, 0, NULL}
};

static struct PyModuleDef finance_module = {
    PyModuleDef_HEAD_INIT,
    "finance",
    "Finance helper module implemented in C.",
    -1,
    FinanceMethods
};

PyMODINIT_FUNC PyInit_finance(void)

```

```
{  
    return PyModule_Create(&finance_module);  
}
```

To create the module run the following instructions from a terminal window:

```
python setup.py build  
pip install .
```

Here's a small piece of Python code to test everything has worked:

```
import finance  
# principal, rate, times_per_year, years  
  
interest = finance.compound_interest(1000, 0.05, 1, 10)  
print(interest)
```
